

MCP 기반 5G 슬라이스 자율 오케스트레이션 설계서

B · 모델 · 정책

TensorFlow 적재 · 정책 래핑 · 자기 개선

담당	② slice-policy · ③ slice-market · ⑤ slice-feedback
구성	공통 계약 + 담당 상세 (—페이지)
저장소	DBAJK/SRM-main-mcp
작성	2026-09-21

이 본은 전체 설계서에서 **B 담당분**을 뽑은 것이다. 앞의 **공통 계약**(공통 타입 · 도구 일람 · 데이터 연쇄 · 제어 루프 · 오류 규약 · 금지 필드)은 **3명이 동일하게 읽는다**. 그 부분을 건너뛰면 자기 서버만 보고 계약을 깬다.

내 범위

② policy	list_policies · propose_allocation · compare_policies · classify_demand
③ market	list_offerings · score_offerings · explain_score · procure · update_rating
⑤ feedback	report_outcome · get_reliability_table

내가 생산하는 값 — 남이 여기에 의존한다

값	출처	소비자	비고
PolicyProposal	② propose_allocation	C · A(④)	allocation · confidence · status
DemandClass	② classify_demand	C · A(④)	지배 수요. 상황 정답이 아님
ScoredOffering	③ score_offerings	C	벤더 5곳 점수 내림차순
capacity_gain	③ procure	A(①) add_capacity	조달을 환경에 반영하는 유일한 값
Outcome	⑤ report_outcome	C · A(④)	sla_met · error · vendor_id
recent_error	⑤ get_reliability_table	② (C가 중계)	②의 LSTM 신뢰도 공급선

내가 소비하는 값 — 없으면 내가 막힌다

값	생산자	쓰는 곳
Observation · HistoryBlock	A(①)	② propose_allocation
situation	C	② propose_allocation — 기본값 없음
current_step	A(①) observation.step	③ procure
decision_id	A(④)	⑤ report_outcome

서버끼리는 직접 대화하지 않는다. 위의 모든 값은 오케스트레이터(C)가 옮겨 심는다. 내 도구가 남의 도구를 부르는 코드는 어디에도 없어야 한다 — 하나라도 있으면 §2.2 규약 위반이고, 무상태 서버가 상태를 갖게 된다.

공통 타입

공통 계약 · `claude/spec/common.md`

모든 서버가 공유한다. `mcp/common/schema.py` 에 pydantic 모델로 한 번만 정의한다.

SliceTriple

3개 슬라이스에 대한 실수 3개. **배열이 아니라 딕셔너리다.** (배열은 인덱스 순서가 바뀌면 조용히 틀린다)

```
{"embb": 0.40, "urllc": 0.40, "mmtc": 0.20}
```

SliceFlags

```
{"embb": true, "urllc": true, "mmtc": false}
```

열거형

이름	값	비고
Situation	"normal" · "emergency" · "special_event" · "iot_surge"	에이전트가 관측으로부터 추론해 만드는 값. 어떤 도구도 알려주지 않는다
PolicyName	"rule_based" · "lstm_forecast" · "dqn"	
SliceType	"eMBB" · "URLLC" · "mMTC"	③에서만 쓴다. 대소문자가 <code>vendors.json</code> 의 키와 정확히 일치해야 한다
Status	"ok" · "unavailable" · "error"	

숫자 타입 규약 ⚠

JSON은 numpy를 모른다. 모든 경계에서 파이썬 기본형으로 변환한다.

타입	<code>json.dumps()</code>	비고
<code>np.ndarray</code>	실패	<code>.tolist()</code> 필요
<code>np.bool_</code>	실패	<code>bool()</code> 필요
<code>np.float64</code>	통과	파이썬 <code>float</code> 상속. 우연히 통과하는 것
<code>float</code> , <code>bool</code> , <code>int</code>	통과	

`violations` 가 정확히 `np.bool_` 이다 (`ml_orchestrator_demo.py:563`). 그대로 반환하면 도구 호출이 예외로 죽는다. `np.float64` 는 통과하므로 한참 잘 돌다가 `violations` 에서만 터진다.

도구 일람

공통 계약 · `claude/spec/tools.md`

5개 서버 · 도구 21개. 상세 계약은 `spec/<server>.md`.

서버	도구	입력	출력	부작용
① observe	get_observation	—	Observation	없음
	step	n	Observation	상태 전진
	apply_allocation	SliceTriple	ApplyResult	배분 변경
	get_history	n	HistoryBlock	없음
	add_capacity	slice_type, amount, expires_at_step, slice_id	CapacityState	용량 증설
	reset	run_id, scenario, seed	Observation	에피소드 초기화
② policy	list_policies	—	[PolicyInfo]	없음
	propose_allocation	policy, Observation, situation, history, recent_error	PolicyProposal	없음
	compare_policies	Observation, situation, history, recent_errors	[PolicyProposal]	없음
	classify_demand	Observation	DemandClass	없음
③ market	list_offerings	slice_type, region	[Offering]	없음
	score_offerings	slice_type, qos	[ScoredOffering]	없음
	explain_score	vendor_id, slice_type, qos	ScoreBreakdown	없음
	procure	vendor_id, slice_type, qos, duration_steps, current_step	Procurement	조달 기록
	update_rating	vendor_id, outcome	RatingUpdate	vendors.json 갱신
④ audit	record_decision	판단 일체	{decision_id}	기록
	record_escalation	판단 + 사유	{escalation_id, decision_id, fallback}	기록 = 개입 1회 + 폴백 결정
	get_decisions	n, kind	[Decision]	없음
	get_metrics	window	Metrics	없음

서버	도구	입력	출력	부작용
⑤ feedback	report_outcome	decision_id , Observation	Outcome	신뢰도 갱신
	get_reliability_table	—	ReliabilityTable	없음

부작용 있는 도구 8개는 한 스텝에 정해진 횟수만 호출된다.

step · apply_allocation · record_decision · report_outcome 은 각 1회,
record_escalation · procure · add_capacity 는 0 또는 1회.

1스텝 표준 호출 순서

```

1. ①.get_observation()                → obs
1b. ①.get_history(10)                 → history      ← 빠뜨리면 LSTM이 영영 unavailable
2. ②.classify_demand(obs)              → dominant      (선택)
   ⑤.get_reliability_table()           → effective, recent_error
   [LLM] situation · confidence.situation 추론
3. ②.propose_allocation(policy, obs, situation, history, recent_error)
   [파이썬] combined = (situation × intrinsic × effective)^(1/3)

   ┌ combined < 0.45 → ④.record_escalation(...)
   │                 → escalation_id, decision_id, fallback_allocation
   │                 **4.5를 건너뛰고 6으로 ** allocation ← fallback_allocation
   │                 (record_decision을 또 부르면 같은 스텝에 decision 레코드가 둘)
   └ 계속 ↓

4. [LLM 조달 판단 · demand_pressure ≥ 1.0일 때만]
   ③.score_offerings → ③.procure(current_step=obs.step) → ①.add_capacity
5. ④.record_decision(..., slice_id, vendor_id) → decision_id      (apply 전에!)

6. ①.apply_allocation(**allocation)      → normalized, delta
7. ①.step()                             → obs+1
8. ⑤.report_outcome(decision_id, obs+1)   → sla_met, vendor_id
9. [vendor_id 있으면] ⑤.update_rating(vendor_id, {sla_met, decision_id})

```

조달이 기록보다 앞서는 이유 — record_decision 이 slice_id · vendor_id 를 인자로 받기 때문이다
(설계서 §3.1 규칙 4). 뒤집으면 결정 레코드에 vendor_id 가 없고, ⑤ report_outcome 이 그 레코드에서
vendor_id 를 찾으므로 항상 null 이 되어 ⑤→③ 레이팅 되먹임이 통째로 죽는다. "판단했으나 실행하지 못한
경우를 남긴다"는 규칙 3의 목적은 훼손되지 않는다 — 조달 실패도 slice_id: null + rationale 로 기록되기
때문이다.

add_capacity 는 반드시 step() (7) 전에 부른다. 뒤로 가면 용량이 다음 스텝부터 반영되어 조달한 바로 그
스텝은 효과를 못 본다 (W1).

1b를 빠뜨리면 lstm_forecast 가 영영 status: "unavailable" (history_insufficient)이 되어 정책 선택
축이 죽는다. history 는 3단계의 인자인데 얻는 호출이 순서에 없으면 null 로 넘어간다.

에스컬레이션 분기는 4·5를 건너뛰다. `record_escalation` 이 이미 `decision_id` 와 `fallback_allocation` 을
내므로 `record_decision` 을 또 부르면 같은 스텝에 `kind: "decision"` 레코드가 둘 생기고
`sum(policy_usage)` 불변식이 깨진다 (`spec/audit.md`).

루프는 파이썬이 돌린다. LLM은 2·4단계에서만 호출하며 답하는 값은 넷뿐이다 — `situation` ·
`confidence.situation` · 정책 선택 · 조달 여부. `combined` 와 에스컬레이션 여부는 파이썬이 계산한다. 근거는
`flow/loop.md` .

서버끼리 직접 대화하지 않는다. 모든 값은 에이전트를 통과한다. 전체 도식과 출력→입력 대응표는 `flow/data-chain.md` .

제어 루프 계약

공통 계약 · `claude/flow/loop.md`

1스텝의 시간 규약 · 액추에이터 · 고정 루프 · 컨텍스트 · 에스컬레이션 정의

3. 제어 루프 계약

분리 설계서에 없던 부분이고, 측정 지표의 정의가 전부 여기에 달려 있다.

3.1 시간 규약

원 코드의 `run_step()` (:535)은 한 스텝 안에서 트래픽 생성 → 이용률 계산 → 배분 갱신을 모두 한다. MCP에서는 배분 갱신이 에이전트 쪽으로 나가므로 이 순서를 쪼개야 하고, 쪼개는 지점이 곧 지표의 정의가 된다.

```
# ml_orchestrator_demo.py:545
utilization = self.update_utilization(traffic)    # ← 이 시점의 allocation은 직전 스텝의 것
...
allocation = self.update_allocation*(...)        # ← 그 다음에 갱신
```

즉 이용률은 언제나 "직전 결정의 결과" 다. 이 성질을 그대로 살린다.

```
reset(run_id, scenario, seed)
└─ alloc ← [0.4, 0.4, 0.2], cap ← [1, 1, 1], traffic. 생성
   util. = traffic. / (alloc × cap)

— 스텝 t —
1. obs_t = ①.get_observation()           # util_t 는 a_{t-1}의 성적표
2. [에이전트 판단] ②.classify_demand / propose_allocation
   ⑤.get_reliability_table
2b. [조달 분기] obs_t.demand_pressure ≥ 1.0 일 때만
   ③.score_offerings → ③.procure(..., current_step=t)
   ④.add_capacity(amount=capacity_gain, expires_at_step)
3. ④.record_decision(..., slice_id, vendor_id) → decision_id
4. ①.apply_allocation(a_t)               # 액추에이터. 평활·클립·정규화
5. ①.step()                             # 만료 회수 → traffic_{t+1}
   # util_{t+1} = traffic_{t+1} / (a_t × cap_{t+1})
6. ⑤.report_outcome(decision_id, obs_{t+1})
```

따르는 규칙 네 가지

- 결정의 채점은 1스텝 지연된다. t의 결정은 t+1의 관측으로 평가된다. `report_outcome` 은 반드시 `step()` 이후에 호출한다.
- 에피소드 마지막 결정은 채점되지 않는다. 지표 집계에서 제외한다. 60스텝이면 유효 표본 59.
- 기록은 배분 실행(4)보다 먼저다. 에이전트가 판단만 하고 실행에 실패한 경우도 ④에 남는다. 실행 후 기록이면 실패 사례가 사라져 자율 처리율이 과대평가된다.

4. 조달(2b)만 기록보다 앞선다. 규칙 3의 예외이며, `record_decision` 이 `slice_id` · `vendor_id` 를 인자로 받기 때문이다.

조달이 기록보다 앞서는 것이 규칙 3과 충돌하지 않는 이유

규칙 3의 목적은 "판단했으나 실행하지 못한 경우를 남기는 것"이다. 조달은 판단의 일부로서 기록에 포함되므로 목적이 훼손되지 않는다.

```
조달 성공 → record_decision(slice_id="slice-...", vendor_id="vendor-1")
조달 실패 → record_decision(slice_id=null, rationale="procure rejected: capacity cap exceeded")
```

실패해도 `rationale` 에 남는다. 조달을 시도했다는 사실 자체가 판단의 증거이므로 기록되어야 한다.

`add_capacity` 는 반드시 `step()` (5) 전에 호출한다. 뒤로 가면 용량이 다음 스텝부터 반영되어 조달한 바로 그 스텝은 효과를 보지 못한다. 에이전트 입장에서 "샀는데 아무 일도 안 일어났다"가 되어 조달 학습이 왜곡된다.

3.2 `apply_allocation` 은 액추에이터다

현재 평활·클립·정규화가 `update_allocation_ml ( :401~407 )`과 `update_allocation_rule_based ( :459~462 )`에 중복되어 있다.

```
stability_factor = 0.7
new_allocation = 0.7 * self.allocation + 0.3 * target
new_allocation = np.clip(new_allocation, 0.1, 0.8)
new_allocation = new_allocation / np.sum(new_allocation)
```

이걸 정책에서 떼어 ④의 `apply_allocation()` 한 곳에 둔다. 세 가지 이득이 있다.

- 정책 비교가 공정해진다 — 모든 정책이 동일한 액추에이터를 통과한다. 정책은 "목표 배분"만 내고, 물리적 제약(급변 금지, 슬라이스당 10~80%)은 환경이 강제한다. 현실의 RAN 제약에 대응하므로 모델링으로도 타당하다.
- 에이전트가 제약을 학습할 수 있다 — 요청이 얼마나 깎였는지 `accepted / normalized / reason` 으로 돌려준다.
- DQN 추가 시 재작성이 없다 — DQN은 이산 액션 27개를 배분 벡터로 바꾸기만 하면 된다.

```
apply_allocation(embb, urllc, mmtc) -> {
    "accepted": bool,
    "requested": {"embb": 0.20, "urllc": 0.70, "mmtc": 0.10},
    "normalized": {"embb": 0.34, "urllc": 0.49, "mmtc": 0.17},    # 평활 후
    "reason": "smoothed(0.7) + clipped to [0.1,0.8]"
}
```

`accepted=False` 는 입력이 비정상일 때만(음수, 합이 0, NaN). 정상 입력은 항상 수용하되 변형해서 알린다.

3.3 오케스트레이터는 고정 루프다 — LLM이 순서를 정하지 않는다

MCP는 도구를 줄 뿐 호출 순서를 강제하지 않는다. §3.1의 6단계를 LLM에게 맡기면 스텝당 6~10회 호출 × 120스텝 × 60회 실행 동안 반드시 다음이 일어난다.

- `report_outcome` 을 빠뜨린다 → 그 스텝이 채점되지 않는다 (§1.5 V2와 같은 증상)
- 순서가 뒤집힌다 → `add_capacity` 가 `step()` 뒤로 가면 조달이 무효가 된다 (§1.6 W1)
- 20~30스텝 뒤 드리프트한다 → 후반부 데이터가 전반부와 다른 절차로 생산된다

§8의 오류 규약이 잡는 것은 일부뿐이다. 대부분의 위반은 조용하다 — 호출하지 않은 것은 오류를 내지 않는다.

따라서 루프는 파이썬이 돌리고, LLM은 판단 지점에서만 호출한다.

```
for t in range(total_steps):
    obs = observe.get_observation()
    rel = feedback.get_reliability_table()

    j = ask_llm(obs, rel, recent)          # ← LLM은 여기서만
    #   j.situation, j.confidence_situation
    #   j.policy, j.procure_or_not, j.escalate_or_not

    if j.procure: ...                      # 2b
    audit.record_decision(...)             # 3   ← 파이썬이 보장
    observe.apply_allocation(...)          # 4
    observe.step()                         # 5
    feedback.report_outcome(...)           # 6
```

LLM이 답할 것은 네 가지뿐이다 — `situation` / 어느 정책 / 조달할까 / 사람을 부를까. §1.5에서 “*AGENT* 가 생산자인 값은 네 개뿐이어야 한다” 고 한 그 넷과 정확히 일치한다.

이것이 논문에도 유리하다. 자율성 주장은 판단에 관한 것이지 도구 호출 순서에 관한 것이 아니다. 루프를 고정하면

- 변수가 격리되어 비교군 간 차이가 전부 판단에서 나온다
- “에이전트가 루프를 잘못 돌아서 SLA가 나뻐다” 는 반론이 원천 차단된다
- 60회 실행이 동일한 절차로 생산된다

이 구조에서도 에이전트는 여전히 자율적이다. 무엇을 할지는 LLM이 정하고, 정한 것을 빠짐없이 실행하는 것만 파이썬이 보장한다. 사람으로 치면 절차를 지키는 것과 판단하는 것의 분리다.

3.4 컨텍스트는 스텝마다 재구성한다

120스텝 대화를 누적하면 후반에 100k 토큰을 넘는다. 비용·지연이 폭증하고, 모델이 20스텝 전의 낡은 관측에 주의를 뺏긴다.

매 스텝 프롬프트를 새로 만든다. 대화 이력을 이어붙이지 않는다.

시스템 프롬프트 (고정, minimal/advisory 2벌)

+ 현재 관측 1건

+ ④의 신뢰도 표

+ 최근 N스텝 요약 (N = 5, 한 줄씩: step/situation/policy/sla_met)

최근 이력을 **원본이 아니라 요약**으로 넣는 것이 핵심이다. `get_decisions(n=5)` 를 그대로 넣으면 관측이 통째로 달려와 5배가 된다 (TOOLS.md §9).

부수 효과로 프롬프트가 재현 가능해진다. 같은 스텝에서 같은 입력이면 같은 프롬프트가 만들어지므로, 프롬프트 해시를 `decisions.json` 에 기록해 사후 추적할 수 있다 (§9-9).

3.5 에스컬레이션의 조작적 정의

분리 설계서 §6의 "개입 1회"를 도구 수준으로 확정한다.

```
④.record_escalation(observation, reason, confidence) -> {"escalation_id": str}
```

- 이 도구를 호출한 것 자체가 개입 1회다. 별도의 사람 응답 대기는 없다.
- 호출 후 에이전트는 **rule_based + situation="normal"** 로 진행한다. 이것이 "사람이 올 때까지의 안전 기본 동작"이며, 사람의 개입 없이도 에피소드가 끝까지 돌아간다.
- 따라서 모든 비교군이 동일한 스텝 수를 소화한다. SLA 위반 비교가 공정해진다.

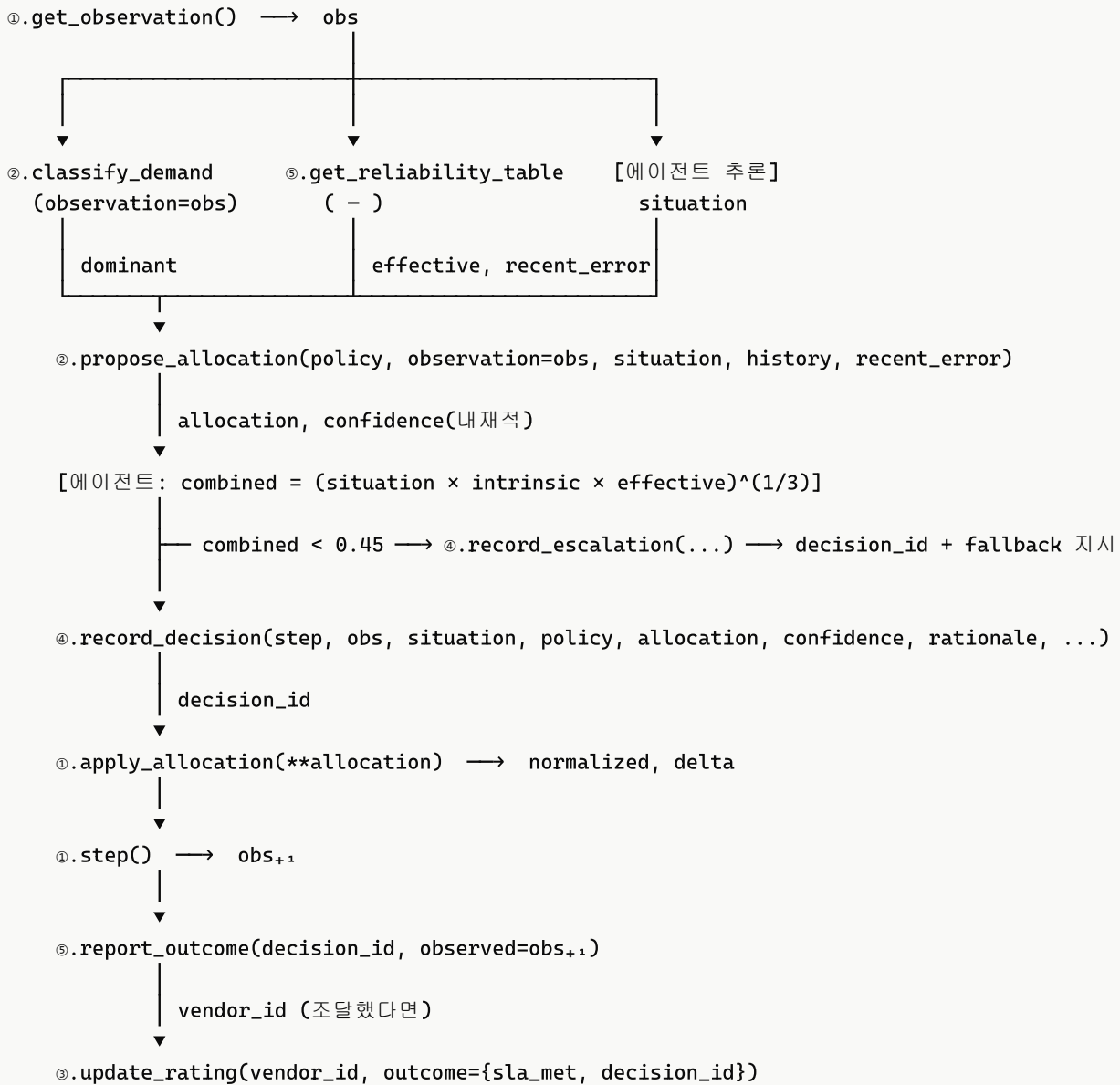
이 정의 덕분에 실험에 사람이 실제로 필요 없다. "개입 횟수"는 사람을 부른 횟수이지 사람이 일한 시간이 아니다 — 이 점을 논문에 명시한다.

데이터 연쇄 — 어떤 출력이 어떤 입력이 되는가

공통 계약 · `claude/flow/data-chain.md`

서버끼리 직접 대화하지 않는다. 모든 값은 에이전트를 통과한다.

1스텝 흐름



조달 분기

`demand_pressure ≥ 1.0` 일 때만 의미가 있다. 배분 결정과 같은 스텝, `step()` 전에 일어난다.

```

obs.demand_pressure ≥ 1.0
    ↓
③.score_offerings(slice_type, qos) → vendor_id (1위)
    ↓
⑤.procure(vendor_id, slice_type, qos, duration_steps, current_step=obs.step)
    |
    capacity_gain, expires_at_step, slice_id
    ↓
①.add_capacity(slice_type, amount=capacity_gain, expires_at_step, slice_id)
    |
    demand_pressure (반영 후)
    ↓
[에이전트: 여전히 ≥ 1.0이면 추가 조달 또는 위반 감수]
slice_id · vendor_id → ④.record_decision에 함께 기록

```

③과 ①은 직접 대화하지 않는다. capacity_gain 을 에이전트가 옮겨 심는다. ⑤ → ③의 update_rating 중계, ⑤ → ②의 recent_error 중계와 같은 패턴이다.

출력 → 입력 대응표

생산	필드	소비	파라미터
①. <code>get_observation</code>	<code>Observation</code> 전체	②. <code>propose_allocation</code>	<code>observation</code>
		②. <code>classify_demand</code>	<code>observation</code>
		④. <code>record_decision</code>	<code>observation</code>
	<code>demand_pressure</code>	—	에이전트의 조달 판단 근거
①. <code>get_history</code>	<code>HistoryBlock</code> 전체	②. <code>propose_allocation</code>	<code>history</code>
①. <code>step</code>	<code>observation</code>	⑤. <code>report_outcome</code>	<code>observed</code>
②. <code>propose_allocation</code>	<code>allocation</code>	①. <code>apply_allocation</code>	<code>embb / urllc / mmtc</code>
	<code>confidence</code>	④. <code>record_decision</code>	<code>confidence.intrinsic</code>
	<code>in_distribution</code>	④. <code>record_decision</code>	<code>in_distribution</code>
②. <code>classify_demand</code>	출력 전체	④. <code>record_decision</code>	<code>demand_class</code>
②. <code>compare_policies</code>	탈락 후보 목록	④. <code>record_decision</code>	<code>considered</code>
③. <code>score_offerings</code>	<code>vendor_id</code>	③. <code>explain_score</code> · ③. <code>procure</code>	<code>vendor_id</code>
③. <code>procure</code>	<code>slice_id</code>	④. <code>record_decision</code> · ①. <code>add_capacity</code>	<code>slice_id</code>
	<code>vendor_id</code>	④. <code>record_decision</code>	<code>vendor_id</code>
	<code>capacity_gain</code>	①. <code>add_capacity</code>	<code>amount</code>
	<code>expires_at_step</code>	①. <code>add_capacity</code>	<code>expires_at_step</code>
④. <code>record_decision</code>	<code>decision_id</code>	⑤. <code>report_outcome</code>	<code>decision_id</code>
④. <code>record_escalation</code>	<code>decision_id</code>	⑤. <code>report_outcome</code>	<code>decision_id</code>
	<code>fallback_allocation</code>	①. <code>apply_allocation</code>	<code>embb / urllc / mmtc</code>
⑤. <code>report_outcome</code>	<code>vendor_id</code> , <code>sla_met</code>	③. <code>update_rating</code>	<code>vendor_id</code> , <code>outcome</code>
⑤. <code>get_reliability_table</code>	<code>{policy}.effective</code>	④. <code>record_decision</code>	<code>confidence.empirical</code>
	<code>{policy}.recent_error</code>	②. <code>propose_allocation</code>	<code>recent_error</code>
	테이블 전체	②. <code>compare_policies</code>	<code>recent_errors</code>
에이전트	<code>run_id</code>	①. <code>reset</code> · ④ 전체	<code>run_id</code>

에이전트가 만들어내는 값

어느 도구에서도 오지 않는다. LLM이 답하는 값은 정확히 넷이다. 어느 서버에도 이 코드가 없다.

값	쓰이는 곳	근거
situation	②. propose_allocation , ④. record_decision	관측(특히 트래픽 구성비)으로부터 추론
confidence.situation	④. record_decision	상황 판단 자체에 대한 확신. 어느 도구도 주지 않는다
정책 선택	②. propose_allocation 의 policy	신뢰도 비교
조달 여부	③. procure 호출 여부	demand_pressure + 비용 판단

나머지 둘은 파이썬이 기계적으로 계산한다. LLM에게 묻지 않는다.

값	계산
confidence.combined	(situation × intrinsic × empirical)^(1/3)
에스컬레이션 여부	combined < 0.45

check_contract.py 는 AGENT 생산 값이 정확히 이 넷인지 검사한다 — 늘면 판단이 서버로 샌 것이고, 줄면 기여가 사라진 것이다. (flow/loop.md — 루프는 파이썬이 돌리고 LLM은 판단 지점에서만 호출한다)

오류 규약

공통 계약 · `claude/flow/errors.md`

원칙: 에이전트가 판단으로 대응할 수 있는 것은 값으로, 코드 버그는 예외로.

값으로 돌려줄 것 vs 예외를 던질 것

상황	처리	이유
정책 사용 불가 (이력 부족, 모델 없음)	<code>status</code> 필드로 반환	에이전트의 판단 재료
배분 입력이 비정상	<code>accepted: false</code> 로 반환	에이전트가 재시도 가능
존재하지 않는 <code>vendor_id</code>	<code>error</code> 필드로 반환	오타 복구 가능
존재하지 않는 <code>decision_id</code>	<code>error</code> 필드로 반환	
용량 상한 초과 (<code>add_capacity</code>)	<code>accepted: false</code> 로 반환	에이전트가 다른 슬라이스를 택할 수 있음
<code>current_step</code> 이 과거 (<code>procure</code>)	<code>status: "rejected"</code> 로 반환	순서 실수. 복구 가능
잘못된 열거형 값	예외 (FastMCP가 스키마 검증)	계약 위반
파일 I/O 실패	예외	복구 불가

오류 메시지도 프롬프트의 일부다. 복구에 필요한 정보를 같이 준다.

```
// 존재하지 않는 벤더 - available을 같이 주면 에이전트가 한 번에 복구한다
{"error": "unknown_vendor", "vendor_id": "vendor-9",
 "available": ["vendor-1", "vendor-2", "vendor-3", "vendor-4", "vendor-5"]}
```

호출 순서 위반

위반	반환
<code>report_outcome</code> 을 <code>step()</code> 전에 호출	<code>{"error": "premature_scoring", "reason": "decision at step 12, observed step 12; expected >= 13"}</code>
<code>add_capacity</code> 를 <code>step()</code> 후에 호출	허용되지만 한 스텝 늦게 반영된다. 조달한 스텝은 효과를 못 본다
<code>procure</code> 없이 <code>add_capacity</code> 호출	허용. 용량은 늘지만 비용이 기록되지 않아 지표가 왜곡된다. 검사기가 경고
<code>apply_allocation</code> 없이 <code>step()</code> 두 번	허용. 직전 배분이 유지됨
<code>reset()</code> 없이 시작	허용. <code>seed=0</code> , <code>scenario="normal"</code> 로 자동 초기화

금지 필드 검사

공통 계약 · `claude/flow/forbidden.md`

`baseline` arm을 제외한 모든 실행에서 아래 문자열이 에이전트 컨텍스트에 0건이어야 한다. 도구 설명·반환값·오류 메시지 전부가 검사 대상이다. 1건이라도 나오면 그 실행은 폐기한다.

```
FORBIDDEN = ["is_emergency", "is_special_event", "is_iot_surge",
              "ground_truth", "perception_accuracy", "escalation_precision"]
```

출처	어디로만 나가나
<code>is_emergency</code> · <code>is_special_event</code> · <code>is_iot_surge</code>	①이 <code>runs/{run_id}/truth.jsonl</code> 에만 기록. 어떤 도구도 읽지 않는다
<code>perception_accuracy</code> · <code>escalation_precision</code>	<code>eval/score.py</code> 가 <code>truth.jsonl</code> 과 조인해 오프라인 계산. ④ <code>get_metrics</code> 는 반환하지 않는다

baseline arm의 예외

`baseline`만 `truth.jsonl`을 읽는다. 기존 시스템(CLI `--emergency`)의 재현이므로 정당하지만, 코드 경로를 물리적으로 분리한다. 같은 함수에 `if arm == "baseline"`으로 두면 실수로 다른 비교군에 샌다.

situation ↔ truth 매핑 규칙

`eval/score.py`가 `perception_accuracy`를 계산하려면 4값 열거형과 3개 `bool` 사이의 대응을 사전에 못박아야 한다. 채점 후에 정하면 유리한 쪽으로 해석할 수 있다.

truth	정답 situation
전부 <code>false</code>	<code>normal</code>
<code>is_emergency</code> 만 <code>true</code>	<code>emergency</code>
<code>is_special_event</code> 만 <code>true</code>	<code>special_event</code>
<code>is_iot_surge</code> 만 <code>true</code>	<code>iot_surge</code>
둘 이상 <code>true</code>	발생하지 않음

시나리오 정의상 세 플래그는 상호 배타적이다. `reset(scenario=...)`은 하나만 켜고, `mixed`의 전환 시점(`test_scenarios.py:92`)도 겹치지 않는다 — 스텝 20~50 `special_event`, 60~90 `emergency`, 100~120 `iot_surge`이며 그 사이는 전부 `normal`이다.

그래도 ①이 `truth.jsonl`을 쓸 때 상호 배타성을 `assert` 한다. 시나리오를 나중에 추가하다 겹치면 채점 기준이 조용히 무너진다.

전환 경계 처리

플래그가 바뀐 그 스텝의 결정은 정답 판정에서 제외한다. 에이전트는 `obs_t` 를 보고 판단하는데 `obs_t` 는 전환 직전의 트래픽을 반영하므로, 전환 스텝을 미탐으로 세면 원리적으로 맞힐 수 없는 것을 틀렸다고 세는 셈이다. `mixed` 120스텝에서 제외 대상은 5스텝이다.

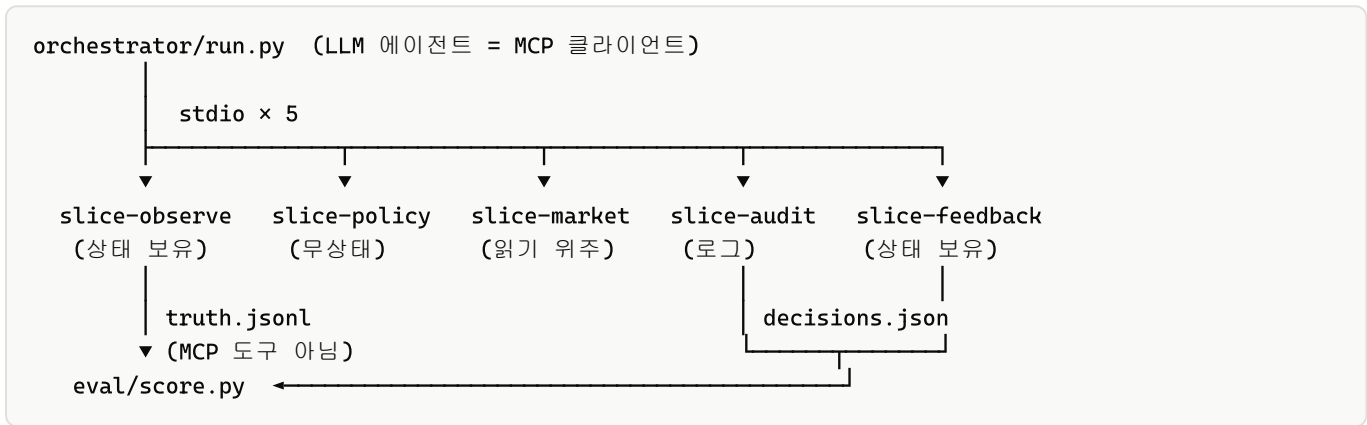
런타임 구조

공통 계약 · `claude/build/runtime.md`

프로세스 구성 · 저장소 레이아웃 · 기술 스택

2. 런타임 구조

프로세스 구성



- 전송: stdio. 단일 머신 연구 프로토타입이므로 HTTP 불필요. 서버당 독립 프로세스.
- ①만 에피소드 생명주기를 가진다. 나머지 넷은 프로세스가 죽어도 파일에서 복구된다.
- 동시 쓰기 없음: 에이전트가 도구를 순차 호출한다. 락을 두지 않는다 (분리 설계서 §2.1 승계).
- 단, 쓰기는 원자적으로 한다 — `tmp` 파일에 쓴 뒤 `os.replace()`. 중단 시 JSON 파일이 깨지는 것만 막으면 충분하다.

저장소 레이아웃

```
Build-A-Thon-SRM-main/
├── mcp/
│   ├── common/
│   │   ├── paths.py          # 경로 상수 한 곳
│   │   ├── schema.py        # Observation, Decision 등 pydantic 모델
│   │   └── store.py          # 원자적 JSON 읽기/쓰기
│   ├── observe/
│   │   ├── server.py         # ① FastMCP
│   │   └── env.py            # MLOrchestrator에서 추출한 순수 시뮬레이터
│   ├── policy/
│   │   ├── server.py         # ②
│   │   ├── descriptions.py   # 도구 설명 2벌 (§6.3)
│   │   ├── rule.py lstm.py dqn.py classify.py
│   ├── market/
│   │   ├── server.py         # ③
│   │   └── scoring.py        # engine.py에서 추출
│   ├── audit/
│   │   ├── server.py         # ④
│   │   └── metrics.py        # 관측 가능 지표만
│   ├── feedback/
│   │   ├── server.py         # ⑤
│   │   └── reliability.py    # EMA 갱신 (§6.2)
├── orchestrator/
│   ├── run.py                # 에이전트 루프 + 비교군 스위치
│   └── prompts/
├── eval/
│   ├── score.py              # 정답 조인 → 정답 의존 지표
│   └── plots.py
├── data/
│   ├── vendors.json          # 실행에 걸쳐 유지
│   └── vendors.json          # ⑥ 소유 (부트스트랩 시 모델 A 복사)
├── runs/{run_id}/
│   ├── decisions.json        # 실행마다 새로 (§5.0)
│   ├── decisions.json        # ④ 기록 · ⑤ 추가기입
│   ├── truth.jsonl           # ① 기록, 도구 미노출
│   ├── reliability.json      # ⑤ (cold 모드)
│   └── context/              # C의 누출 검사 덤프
├── tools/
│   └── step0_verify_models.py # 0단계
```

기존 코드는 건드리지 않는다. `mcp/` 는 `ml_orchestrator_demo.py` · `engine.py` 에서 로직을 추출(copy-and-adapt) 하지 저 파일들을 import 하지 않는다. 이유: 두 파일 다 최상단에서 무거운 부작용을 일으킨다 (`MLOrchestrator.__init__` 이 `results/` 폴더 생성, 모듈 레벨 `TF import`). 추출 대상은 §4에 위치를 명시했다.

기술 스택

항목	선택	근거
MCP 프레임워크	<code>fastmcp</code>	데코레이터 기반. 서버당 100줄 내외
Python	3.10	TF 2.15 호환 상한
TensorFlow	2.15.1 고정	Keras 2. SavedModel 직접 적재 (정정 I)
스키마	<code>pydantic</code> v2	FastMCP가 타입힌트에서 JSON Schema 자동 생성
가상환경	신규 생성	동봉된 <code>venv/</code> 는 Python 3.14 + 타 PC 절대경로. 사용 불가

공유 데이터 스키마

공통 계약 · claude/build/store.md

실행 생명주기 · decisions.json · reliability.json · truth.jsonl · vendors.json

5. 공유 데이터 스키마

5.0 실행 생명주기 — 무엇이 언제 초기화되는가

`reset()` 은 ①만 초기화한다. ②③④⑤의 상태는 그대로 남는다. 비교군 4 × 시나리오 5 × 시드 3 = 60회를 돌리는데 산출물 경로가 단일 파일이면 전부 덮어쓰거나 뒤섞인다.

실행 단위로 경로를 나눈다.

data/	← 실행에 걸쳐 유지
vendors.json	③ 소유. 레이팅이 누적된다
reliability.json	⑤ 소유. warm 모드에서만 사용
runs/{run_id}/	← 실행마다 새로
decisions.json	④ 생성 · ⑤ 추가가입
truth.jsonl	④ 기록. 도구 미노출
reliability.json	⑤. cold 모드에서만 사용
context/	C의 누출 검사용 덤프

서버	상태	실행 시작 시	근거
①	배분·용량·이력·리스	<code>reset()</code> 이 전부 초기화	에피소드가 곧 ①의 수명
②	없음 (모델만 상주)	초기화 불필요	무상태
③	vendors.json 의 rating	초기화 안 함 (기본)	레이팅 누적이 자기 개선의 증거
④	runs/{run_id}/decisions.json	첫 record_decision 이 파일 생성	별도 도구 불필요
⑤	reliability.json	모드에 따라 다름	아래

⑤와 ③의 초기화는 실험 변수다.

모드	⑤ reliability.json	③ vendors.json	보이는 것
cold	runs/{run_id}/ — 실행마다 새로	부트스트랩 재실행	단일 에피소드 내 적응
warm	data/ — 전체 누적	누적	장기 자기 개선 (주 결과)

모드는 ⑤·③ 서버 기동 시 환경변수(`SLICE_MEMORY_MODE`)로 정하고, decisions.json 의 config 에 기록한다. 60회 실행을 섞어 돌리면 안 된다 — warm 실행들 사이에 cold 실행이 끼면 누적 레이팅이 오염된다. 모드별로 묶어서 돌린다.

`run_id` 를 발급하는 것은 에이전트(C)다. 형식 `{arm}-{scenario}-s{seed}` . ①의 `reset()` 과 ④의 모든 기록에 같은 값이 들어가야 `truth.jsonl` 과 `decisions.json` 이 `step` 으로 조인된다 (§1.5 V3).

runs/{run_id}/decisions.json

④가 생성하고 ⑤가 같은 레코드에 덧붙인다 (분리 설계서 §2.1).

```
{
  "run_id": "exp-proposed-emergency-s0",
  "config": {"scenario": "emergency", "seed": 0, "arm": "proposed",
            "desc_mode": "minimal", "tau": 0.45},
  "decisions": [
    {
      "decision_id": "exp-proposed-emergency-s0-0012",
      "step": 12,
      "kind": "decision",
      "situation": "emergency",
      "chosen_policy": "rule_based",
      "allocation": {"embb": 0.20, "urllc": 0.70, "mmtc": 0.10},
      "confidence": {"intrinsic": 0.72, "empirical": 0.88, "combined": 0.80},
      "rationale": "URLLC util 1.52 > 1.2, eMBB 동반 상승 없음",
      "observation": { ... },

      "outcome": {                                // ← ⑤가 report_outcome에서 추가
        "sla_met": false,
        "error": 0.18,
        "vendor_id": null,
        "scored_at_step": 13
      }
    },
    { "decision_id": "...-0031", "kind": "escalation",
      "situation": "emergency", "reason": "combined 0.31 < tau 0.45",
      "confidence": {...} }
  ]
}
```

- `kind` 로 결정과 에스컬레이션을 한 배열에 둔다. 시간순 재구성이 쉬워진다.
- `config` 를 파일에 박는다. 비교군 4 × 시나리오 5 × 시드 N의 산출물이 섞이지 않는다.
- `outcome` 이 없는 레코드 = 마지막 스텝 또는 실행 실패. 집계에서 제외 (§3.1 규칙 2).

reliability.json (⑤ 소유)

```
{
  "rule_based": {"r": 0.91, "n": 120, "alpha": 0.2},
  "lstm_forecast": {"r": 0.84, "n": 95, "alpha": 0.2}
}
```

에피소드 간 유지 여부가 실험 설계 변수다. §6.2 참조.

runs/{run_id}/truth.jsonl (① 기록, 도구 미노출)

```
{"step": 12, "is_emergency": true, "is_special_event": false, "is_iot_surge": false}
```

data/vendors.json (③ 소유)

모델 A + regions. ⑤는 직접 쓰지 않는다 (분리 설계서 §2.1 승계).

3인 역할 분담 — 공통

공통 계약 · claude/team/roles.md

0. 병렬 개발의 전제

3명이 동시에 손대면 반드시 두 곳에서 막힙니다.

- A의 **Observation** 스키마를 B와 C가 동시에 소비한다. 이게 안 열면 아무도 시작할 수 없습니다.
- C는 ①~⑤가 다 있어야 루프를 돌릴 수 있다. 순진하게 짜면 C는 마지막 주까지 대기합니다.

두 문제를 Day 0 계약 동결과 C가 만드는 목 서버로 풀니다. §1과 §6이 이 문서의 핵심입니다.

1. Day 0 — 3명이 같이 앉아 끝낼 것 (반나절)

이걸 안 하고 흩어지면 3일 뒤에 합쳤을 때 아무것도 안 맞습니다.

1.1 저장소 정리 — 지금 당장 ⚠

현재 저장소에 병렬 개발을 방해하는 것이 둘 있습니다. 확인한 내용입니다.

(1) **.gitignore** 가 UTF-16으로 저장되어 있어 규칙이 먹지 않습니다.

```
$ file .gitignore
.gitignore: Unicode text, UTF-16, little-endian text, with CRLF
```

Git은 **.gitignore** 를 바이트로 읽습니다. UTF-16이면 모든 ASCII 문자 뒤에 `\x00` 이 붙어 패턴이 깨집니다. 실제로 **venv/** 가 무시되는 건 pip이 만든 **venv/.gitignore** 덕분이지 루트 규칙 때문이 아닙니다.

3명이 각자 실행하면 **data/*.json**, **results/**, **.venv310/** 이 전부 커밋 대상으로 잡혀 매 병합마다 충돌합니다. UTF-8로 다시 저장하고 런타임 산출물을 추가하세요.

```
# UTF-8(BOM 없음)로 재작성
printf '%s\n' '__pycache__/' '*.pyc' '*.egg-info/' 'venv/' '.venv*/' 'logs/' '*.env' \
'results/' 'runs/' 'data/decisions.json' 'data/reliability.json' \
'SRM-main-mcp/' > .gitignore
```

models/ 와 ***.h5** 는 넣지 마세요. 이미 추적 중이라 ignore 규칙이 적용되지 않고, 규칙만 보고 "가중치가 없구나" 오해하게 됩니다. 확인 결과 **가중치는 실물입니다** (LFS 스텝 아님, **variables.data-00000-of-00001** = 1.5 MB). B가 clone만 하면 바로 씁니다.

(2) 저장소 안에 저장소가 클론되어 있습니다.

```
srn-main_mcp/
└─ SRM-main-mcp/      ← 중첩 클론. git status에 ?? 로 뜸
    └─ .git
        └─ README.md
```

지우거나 밖으로 옮기세요. 그대로 두면 누군가 `git add .` 할 때 들어갑니다.

(3) 작업 기준 디렉터리 하나로 고정. `D:\pj\SRM-main\SRM-main` 은 옛 사본입니다. 3명 모두 `srn-main_mcp/` 만 씁니다.

1.2 계약 동결 — `mcp/common/`

A가 작성하고, 3명이 같이 읽고, 그 자리에서 확정합니다. 이후 변경은 PR + 3명 합의.

```
mcp/common/
├─ schema.py      # Observation, Decision, PolicyProposal, Outcome (pydantic)
├─ const.py       # 하드코딩 상수 전부
└─ store.py       # 원자적 JSON 읽기/쓰기
```

`const.py` 에 들어갈 값 — 흩어지면 3명이 각자 다른 값을 쓰게 됩니다.

상수	값	출처
THRESHOLDS	<code>{"embb": 0.9, "urllc": 1.2, "mmtc": 0.8}</code>	<code>ml_orchestrator_demo.py:174</code>
INIT_ALLOCATION	<code>{"embb": 0.4, "urllc": 0.4, "mmtc": 0.2}</code>	<code>:171</code>
STABILITY_FACTOR	<code>0.7</code>	<code>:458</code>
ALLOC_CLIP	<code>(0.1, 0.8)</code>	<code>:461</code>
MINUTES_PER_STEP	<code>15</code>	신규 (정정 F)
START_HOUR	<code>0</code>	신규. \$1.7에서 8 → 0으로 재조정
SEQUENCE_LENGTH	<code>10</code>	<code>:190</code>
TAU (에스컬레이션 문턱)	<code>0.45</code>	설계서 \$6.2
EMA_ALPHA	<code>0.2</code>	\$6.2
SHRINK_R0, SHRINK_M	<code>0.5, 5</code>	\$6.2
RATING_DELTA	<code>(+0.05, -0.20)</code>	\$4 ③
FEATURE_COLUMNS	11개 문자열 리스트	<code>:513~526</code>
CAPACITY_BASE	<code>1.6</code>	신규. 평시 여유 확보 (\$1.7)
CAPACITY_MAX	<code>2.6</code>	기본 + 조달 4회분 (\$1.7)
REFERENCE_BANDWIDTH	<code>{eMBB: 1100, URLLC: 500, mMTC: 120}</code>	<code>vendors.json</code> 중앙값 실측
GAIN_SCALE	<code>0.25</code>	신규. 튜닝 가능
COST_PER_STEP	<code>cost × MINUTES_PER_STEP / 60</code>	시간당 단가 → 스텝 환산 (W3)
MEMORY_MODE	<code>"warm"</code>	<code>cold / warm</code> . ③⑤의 실행 간 유지 여부 (W5)
SCENARIO_STEPS	<code>{normal:60, emergency:60, special_event:60, iot_surge:60, mixed:120}</code>	<code>test_scenarios.py:54</code>

TAU 는 실험 전에 확정하고 이후 절대 건드리지 않습니다. 조정하면 개입 횟수를 원하는 값으로 만들 수 있어 결과가 무의미해집니다.

1.3 파일 소유권 — 충돌 방지

한 파일은 한 사람만 수정합니다. 3인 병합 충돌의 90%가 여기서 예방됩니다.

경로	소유	비고
mcp/common/	A	Day 0 이후 변경은 3인 합의
mcp/observe/ , mcp/audit/	A	
eval/ , fixtures/ , tools/record_fixtures.py	A	
mcp/policy/ , mcp/market/ , mcp/feedback/	B	
tools/step0_verify_models.py , tools/bootstrap_vendors.py	B	
mcp/mock/	C	계약의 실행 가능 명세 (§6)
orchestrator/	C	
docs/	공용	섹션 단위로 나눠 쓰기
기존 코드 (ml_orchestrator_demo.py 등)	아무도 수정 안 함	읽기 전용. 로직은 복사해 가져감

마지막 줄이 중요합니다. 기존 파일을 고치면 3명이 같은 995줄 파일을 건드리게 됩니다. `mcp/` 는 **추출(copy-and-adapt)** 이지 import가 아닙니다.

1.4 브랜치 규칙

main

├─ feature/kim

├─ feature/lee

└─ feature/choi

← 통합 , 직접 커밋 금지

← A

← B

← C

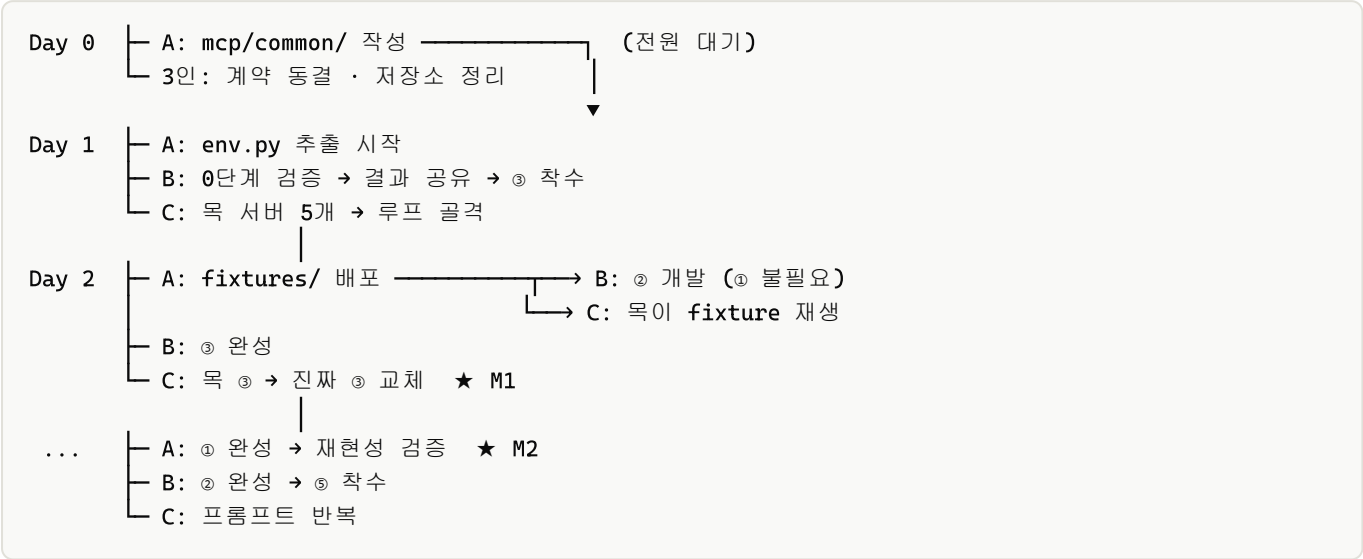
- 합류 지점(§7)마다 `main` 으로 PR. 그 사이에는 각자 브랜치에서 자유롭게.
- `main` 이 깨지면 3명이 동시에 막히므로, **PR 머지 전 최소 조건은 "내 서버가 단독 기동되고 도구 목록이 나온다"** 하나입니다. 테스트 통과까지 요구하면 초반에 진도가 안 나갑니다.

5. 왜 이렇게 묶었나

- A — 실험 인프라를 한 사람이.** `MLOrchestrator` 에서 `run()` 루프를 떼어내는 게 이 프로젝트에서 제일 까다롭습니다 (시뮬레이션·추론·시각화가 995줄에 뒤엉킴). ④ `audit`은 ①의 관측 스키마를 그대로 기록하니 같은 사람이 하면 손발이 맞습니다. 실험 하네스도 ④가 지표를 뽑으니 자연스럽게 따라옵니다.
- B — TensorFlow를 한 사람이.** ②와 ⑤ 모두 "정책"을 알아야 합니다. ⑤의 신뢰도 갱신은 ②가 어떤 정책을 노출하는지에 달려 있어서, 나누면 소통 비용만 늘어납니다. ③ `market`은 가볍게(`engine.py` 거의 그대로) 얹어도 부담이 적습니다.
- C — 판단 로직에 집중.** 오케스트레이터는 코드량은 적은데 반복 튜닝이 많습니다. 프롬프트 고치고 돌려보고 또 고치는 일이라, 서버 개발과 리듬이 다릅니다. 한 사람이 전담하는 게 맞습니다.

부하 균형: A와 B가 비슷하게 큼니다. C는 코드량이 가장 적은 대신 반복 횟수가 많고, 초반에 비는 시간을 목 서버 (C-0)와 누출 검사(C-4)로 채웁니다. 둘 다 C가 아니면 하기 어려운 일입니다.

6. 병렬화 장치 — 누가 누구를 기다리는가



막히지 않는 이유 세 가지

- 1. A의 common/ 이 Day 0에 나온다 — 그 전까지 아무도 못 움직이므로 최우선.
- 2. A의 fixtures/ 가 Day 2에 나온다 — B는 1 없이 2를, C는 1 없이 프롬프트를 개발.
- 3. C의 목 서버가 Day 1에 나온다 — C는 A·B를 기다리지 않고, 동시에 계약 검증 장치가 생김.

교차 검증 — 자기 영역은 자기가 못 본다

검증 대상	검증자	방법
①④의 정답 누출 (정정 D)	C	컨텍스트 덤프 금지어 grep. A는 못 찾음
②의 situation 반영 (정정 E)	A	baseline 과 arm1 의 배분이 정답 일치 시 동일한가
②의 조용한 폴백 (정정 H)	A	④ policy_usage 와 ⑤ reliability 의 n 이 어긋나면 폴백 의심
①의 재현성 (정정 F)	A 자체	동일 시드 2회 → truth.jsonl 바이트 비교

7. 합류 지점

날짜 대신 조건으로 정의합니다. 각 지점에서 main 으로 PR.

	조건	확인 방법
M1 배선	C의 루프가 진짜 ③을 호출해 벤더 5개 점수 수신	목 하나가 진짜로 교체됨
M2 환경	① 동일 시드 2회 → <code>truth.jsonl</code> 동일. <code>fixtures/</code> 배포 완료	<code>cmp</code> 통과
M3 페루프	①②④⑤ 전부 진짜로 1스텝 왕복 + <code>report_outcome</code> 기록	<code>decisions.json</code> 에 <code>outcome</code> 필드 존재
M3.5 계약	<code>check_contract.py</code> 검사 4개 통과	값 유실 0건 (\$1.5)
M4 비교군	4개 arm이 <code>mixed</code> 120스텝 완주. 누출 검사 0건	<code>leak_check.py</code> 통과
M5 결과	60회 실행 + <code>score.py</code> 지표 산출 + 대표 그래프	개입 ↓ / SLA → 확인

M1을 가장 먼저, 가장 작게 만드세요. FastMCP 서버 하나, stdio 연결 하나, 도구 호출 하나가 왕복하면 나머지 20개는 같은 패턴의 반복입니다. ③이 중요해서가 아니라 배선 검증이 목적입니다.

M2에서 반드시 멈추고 확인하세요. 재현이 안 되는 채로 M3~M5를 쌓으면 마지막에 전부 다시 돌려야 합니다.

8. Day 0 체크리스트

셋이 같이 앉아 순서대로.

```
[ ] SRM-main-mcp/ 중첩 클론 제거
[ ] .gitignore UTF-8로 재작성 (models/, *.h5 는 넣지 말 것)
[ ] 작업 기준 디렉터리 srm-main_mcp/ 하나로 합의
[ ] docs/ 에 MCP-decomposition.md · MCP-design.md · ROLES.md 배치 후 커밋
[ ] A/B/C ⇄ kim/lee/choi 브랜치 매핑 확정
[ ] mcp/common/const.py 값 13개 확정 (§1.2) - 특히 TAU = 0.45
[ ] mcp/common/schema.py 확정 - Observation에 is_emergency 계열 부재 확인
[ ] docs/contract.json 초안 - 도구 21개의 requires/produces (C가 작성, 3인 검토)
[ ] run_id 형식 합의: {arm}-{scenario}-s{seed} (V3)
[ ] situation ⇄ truth 매핑·전환 경계 제외 규칙 확정 (V9) - 채점 전에 못 박을 것
[ ] 파일 소유권 표 합의 (§1.3)
[ ] 기존 코드는 아무도 수정하지 않는다 - 합의
[ ] B: Day 1 오전 0단계 착수, 결과 즉시 공유 - 합의
```

마지막 두 줄이 사고를 가장 많이 막습니다.

② `slice-policy` — 계약

담당 B · `claude/spec/policy.md`

무상태. 배분 정책 여러 개를 나란히 노출한다. 설계 근거는 `rationale/policy.md`.

`list_policies()`

입력: 없음 · 출력: `[PolicyInfo]`

필드	타입	설명
<code>name</code>	<code>PolicyName</code>	
<code>description</code>	<code>string</code>	<code>SLICE_DESC_MODE</code> 에 따라 달라짐 (<code>minimal</code> / <code>advisory</code>)
<code>requires</code>	<code>string null</code>	전제조건
<code>available</code>	<code>bool</code>	지금 호출 가능한가
<code>unavailable_reason</code>	<code>string null</code>	불가 사유

TensorFlow 적재에 실패하면 서버는 죽지 않고 해당 정책만 `available: false` 로 내린다.

```
[
  {
    "name": "rule_based",
    "description": "임계값 기반 배분. 상황 라벨을 입력으로 받는다.",
    "requires": null,
    "available": true,
    "unavailable_reason": null,
  },
  {
    "name": "lstm_forecast",
    "description": "시계열 모델 기반 배분. 관측 이력 10스텝 필요.",
    "requires": "history >= 10",
    "available": true,
    "unavailable_reason": null,
  },
  {
    "name": "dqn",
    "description": "강화학습 정책 기반 배분.",
    "requires": "trained weights",
    "available": false,
    "unavailable_reason": "no trained weights"
  }
]
```

`propose_allocation(policy, observation, situation, history, recent_error)`

파라미터	타입	필수	기본	출처
<code>policy</code>	<code>PolicyName</code>	예	—	
<code>observation</code>	<code>Observation</code>	예	—	①. <code>get_observation()</code> 그대로
<code>situation</code>	<code>Situation</code>	예	없음	에이전트가 추론한 상황
<code>history</code>	<code>HistoryBlock</code>	아니오	<code>null</code>	①. <code>get_history()</code> 그대로
<code>recent_error</code>	<code>float</code>	아니오	<code>null</code>	⑤. <code>get_reliability_table()</code> 의 <code>{policy}.recent_error</code>

- **situation** 에 기본값을 두지 않는다. 모든 정책이 필수로 받는다 (배분에 영향은 **rule_based** 만).
- **recent_error** 는 에이전트가 ⑤에서 ②로 중계한다. ②는 무상태라 ⑤를 조회할 수 없다. **null** 이면 보수적 기본값 **0.5** 를 쓰고 **rationale** 에 그 사실을 적는다.

출력: **PolicyProposal**

필드	타입	설명
policy	PolicyName	요청한 정책. 절대 다른 값으로 바꾸지 않는다
allocation	SliceTriple null	목표 배분. 실패 시 null
confidence	float	0~1. 내재적 신뢰도 — 이번 입력에 대한 확신
in_distribution	bool	학습 분포 안인가
status	Status	ok · unavailable · error
reason	string null	status != ok 일 때의 사유
rationale	string	사람이 읽는 근거

폴백 금지 ⚠ — 실패 시 다른 정책을 조용히 호출하지 않는다. **allocation: null** + **status** 로 반환한다. 원본 **update_allocation_ml()** (:341, :414, :419)의 폴백을 옮기지 말 것. 대체 정책 선택은 에이전트의 일이다.

confidence 산출

정책	식
rule_based	$0.50 + 0.30 \times \min(1, \min_i u_i - \theta_i / \theta_i)$
lstm_forecast	$\exp(-3 \cdot \bar{e})$, $\bar{e}$ = recent_error (최근 5회 예측 오차의 EMA, ⑤가 계산)

recent_error 가 **null** 이면 **0.5** 를 쓴다.

예시 1 — 성공

```
// 요청
{"policy": "rule_based",
 "observation": {"step": 12, "utilization": {"embb": 0.624, "urllc": 1.283, "mmtc": 1.015}, ...},
 "situation": "emergency"}
// 응답
{"policy": "rule_based",
 "allocation": {"embb": 0.200, "urllc": 0.700, "mmtc": 0.100},
 "confidence": 0.521,
 "in_distribution": true,
 "status": "ok",
 "reason": null,
 "rationale": "situation=emergency → URLLC 우선 목표 [0.2, 0.7, 0.1]. 여유가 가장 작은 것은 URLLC(1.283/1.2, 7% 초과)라 분기가 자의적 → confidence 0.521."}
```

예시 2 — 이력 부족

```
{
  "policy": "lstm_forecast",
  "allocation": null,
  "confidence": 0.0,
  "in_distribution": false,
  "status": "unavailable",
  "reason": "history_insufficient: 4 < 10",
  "rationale": "시퀀스 길이 미달로 추론하지 않음."
}
```

예시 3 — 모델 적재 실패

```
{
  "policy": "lstm_forecast",
  "allocation": null,
  "confidence": 0.0,
  "in_distribution": false,
  "status": "error",
  "reason": "model_load_failed: SavedModel requires Keras 2 (tensorflow==2.15.1)",
  "rationale": "모델을 적재하지 못해 추론 불가."
}
```

compare_policies(observation, situation, history, recent_errors)

propose_allocation 에서 policy 를 빼고, recent_error 를 정책별 딕셔너리로 바꾼 것.

파라미터	타입	필수	출처
observation	Observation	예	
situation	Situation	예	
history	HistoryBlock	아니오	
recent_errors	{PolicyName: float}	아니오	⑤. get_reliability_table() 을 그대로 넘긴다

출력: [PolicyProposal] — 사용 가능 여부와 무관하게 전 정책을 반환한다.

```
[
  {
    "policy": "rule_based",
    "allocation": {
      "embb": 0.20,
      "urllc": 0.70,
      "mmtc": 0.10
    },
    "confidence": 0.521,
    "status": "ok",
    ...
  },
  {
    "policy": "lstm_forecast",
    "allocation": null,
    "confidence": 0.0,
    "status": "unavailable",
    "reason": "history_insufficient: 4 < 10",
    ...
  },
  {
    "policy": "dqn",
    "allocation": null,
    "confidence": 0.0,
    "status": "unavailable",
    "reason": "no trained weights",
    ...
  }
]
```

호출 비용 주의 — 반환량이 propose_allocation 의 3배. 정책 전환을 고민할 때만 쓰도록 프롬프트에서 유도한다.

classify_demand(observation)

학습된 분류기(TrafficClassifier, 11 → 3 softmax)로 어떤 수요가 지배적인지만 알려준다.

파라미터	타입	필수
observation	Observation	예

출력 필드	타입	설명
dominant	SliceType	최대 확률 클래스
probabilities	{eMBB, URLLC, mMTC: float}	합 1.0
margin	float	1등 - 2등 확률차
available	bool	모델 적재 여부

```
{
  "dominant": "URLLC",
  "probabilities": {
    "eMBB": 0.281,
    "URLLC": 0.644,
    "mMTC": 0.075
  },
  "margin": 0.363,
  "available": true
}
```

이 도구는 "비상 상황인가"를 말하지 않는다. 상황 판단은 에이전트의 몫이다.

③ `slice-market` — 계약

담당 B · `claude/spec/market.md`

외부 벤더 슬라이스 마켓. 설계 근거는 `rationale/market.md`.

qos_requirements 스펙 ⚠

③의 도구 3개(`score_offerings` · `explain_score` · `procure`)가 공통으로 받는다. 코드가 실제로 읽는 키는 이것뿐이다 (`engine.py:339`, `:368`, `:398`):

키	타입	기본값	단위	의미
<code>latency</code>	float	<code>50</code>	ms	요구 지연 상한
<code>bandwidth</code>	float	<code>100</code>	Mbps	요구 대역 하한
<code>reliability</code>	float	<code>99.0</code>	%	요구 신뢰도 하한
<code>advanced_params</code>	object	<code>{}</code>		선택. 1차 범위에서는 쓰지 않음

키 이름에 단위를 붙이지 않는다. `latency_ms` 가 아니라 `latency` 다. 혼동하면 전부 기본값으로 떨어져 모든 벤더가 같은 점수를 받는다.

`advanced_params` 허용 키: `price_sensitivity` (`high|medium|low` → `price_weight` 0.3/0.2/0.1) · `deterministic` · `missionCritical` · `maxPacketSize` · `groupCommunication` · `availability`

list_offerings(slice_type, region)

파라미터	타입	필수	기본	설명
<code>slice_type</code>	<code>SliceType</code>	아니오	<code>null</code>	필터. <code>null</code> 이면 전부
<code>region</code>	string	아니오	<code>null</code>	필터

출력: `[Offering]` — 벤더 5곳 × 슬라이스 3타입. 원본 `data/vendors.json`.

필드	타입	단위
vendor_id · name	string	
slice_type	SliceType	
latency	float	ms
bandwidth	float	Mbps
reliability	float	%
cost	float	시간당
regions	[string]	
rating	float	1.0 ~ 5.0

```
// 요청
{"slice_type": "URLLC"}
// 응답 (실측값, 일부)
[
  {"vendor_id": "vendor-1", "name": "TelcoNet Solutions", "slice_type": "URLLC",
    "latency": 1.0, "bandwidth": 500.0, "reliability": 99.999, "cost": 250.0,
    "regions": ["us-east"], "rating": 4.8},
  {"vendor_id": "vendor-2", "name": "GlobalConnect 5G", "slice_type": "URLLC",
    "latency": 2.0, "bandwidth": 400.0, "reliability": 99.99, "cost": 220.0,
    "regions": ["eu-west"], "rating": 4.5},
  {"vendor_id": "vendor-3", "name": "NextGen Networks", "slice_type": "URLLC",
    "latency": 0.5, "bandwidth": 600.0, "reliability": 99.9995, "cost": 300.0,
    "regions": ["ap-northeast"], "rating": 4.9}
]
```

score_offerings(slice_type, qos_requirements)

파라미터	타입	필수
slice_type	SliceType	예
qos_requirements	object	예

출력: [ScoredOffering] — 점수 내림차순

필드	타입	범위
rank	int	1부터
vendor_id · name	string	
score	float	0 ~ 100
rating	float	1.0 ~ 5.0
cost	float	

점수 공식 (engine.py:719~790 , score_vendor_offering())

```
qos_score = Σ(scorei × weighti) / Σ(weighti)           # 존재하는 기준만, 정규화
reputation = rating / 5.0                                # 가중치 0.2 (URLLC) / 0.1 (그 외)
price      = _calculate_price_score(cost, slice_type)    # 가중치 0.2 (기본)
qos_weight = 1.0 - (reputation_weight + price_weight)
total      = (qos_score × qos_weight + reputation × rep_w + price × price_w) × 100
```

rating 이 총점의 10~20%를 차지한다. ⑤의 피드백이 update_rating 으로 여기에 들어와 순위를 바꾼다.

```
// 요청
{"slice_type": "URLLC", "qos_requirements": {"latency": 1.0, "bandwidth": 400, "reliability": 99.99}}
// 응답 (engine.py 실행 실측값)
[
  {"rank": 1, "vendor_id": "vendor-1", "name": "TelcoNet Solutions", "score": 99.20, "rating": 4.8, "cost": 250.0},
  {"rank": 2, "vendor_id": "vendor-5", "name": "AsiaPacific Telecom", "score": 98.00, "rating": 4.7, "cost": 260.0},
  {"rank": 3, "vendor_id": "vendor-3", "name": "NextGen Networks", "score": 97.60, "rating": 4.9, "cost": 300.0},
  {"rank": 4, "vendor_id": "vendor-4", "name": "EuroSlice Providers", "score": 91.76, "rating": 4.6, "cost": 240.0},
  {"rank": 5, "vendor_id": "vendor-2", "name": "GlobalConnect 5G", "score": 86.20, "rating": 4.5, "cost": 220.0}
]
```

explain_score(vendor_id, slice_type, qos_requirements)

파라미터	타입	필수
vendor_id	string	예
slice_type	SliceType	예
qos_requirements	object	예

출력: ScoreBreakdown

필드	타입	설명
total	float	score_offerings 의 score 와 반드시 일치
criteria_scores	{criterion: float}	기준별 원점수 0~1
weights	{criterion: float}	적용된 가중치
contributions	{criterion: float}	총점 100 기준 기여분
explanation	string	사람이 읽는 요약

구현 규칙 ⚠️ (정정 J) - 기존 `get_score_breakdown()` (engine.py:819)을 호출하지 않는다. `rating` 대신 존재하지 않는 `reputation_score` 키를 읽어 총점이 어긋난다 (99.20 vs 92.00). - `score_vendor_offering()` 의 실제 계산을 분해해 새로 구현한다 (약 30줄). - `neural_network` 키(601바이트, UI용 장식)는 반환하지 않는다.

```
{
  "total": 99.20,
  "criteria_scores": {"latency": 1.00, "bandwidth": 1.00, "reliability": 1.00,
    "reputation": 0.96, "price": 1.00},
  "weights": {"latency": 0.35, "bandwidth": 0.10, "reliability": 0.25,
    "reputation": 0.20, "price": 0.20},
  "contributions": {"latency": 30.00, "bandwidth": 8.57, "reliability": 21.43,
    "reputation": 19.20, "price": 20.00},
  "explanation": "지연 1.0ms가 요구 1.0ms를 충족(1.00). 평판 4.8/5.0이 총점의 19.2점을 기여. 가격 250이 URLLC 기준가와 동일해 감점 없음."
}
```

procure(vendor_id, slice_type, qos_requirements, duration_steps, current_step)

파라미터	타입	필수	범위	설명
vendor_id	string	예		
slice_type	SliceType	예		
qos_requirements	object	예		
duration_steps	int	예	1 ~ 60	시간이 아니라 스텝
current_step	int	예	≥ 0	observation.step . ③은 시뮬레이션 시각을 모른다

비용 환산 — `vendors.json` 의 `cost` 는 시간당, `duration_steps` 는 스텝. `MINUTES_PER_STEP = 15` .

```
cost_total = cost × duration_steps × (MINUTES_PER_STEP / 60)
            = 250 × 10 × 0.25 = 625.0
```

용량 환산 — 상수는 `const.py` .

$\text{capacity_gain} = (\text{offering.bandwidth} / \text{REFERENCE_BANDWIDTH}[\text{slice_type}]) \times \text{GAIN_SCALE}$
 $\text{expires_at_step} = \text{current_step} + \text{duration_steps}$

상수	값
REFERENCE_BANDWIDTH	{eMBB: 1100, URLLC: 500, mMTC: 120} (벤더별 슬라이스 대역폭 중앙값)
GAIN_SCALE	0.25
CAPACITY_BASE / CAPACITY_MAX	1.6 / 2.6 (슬라이스당, ①이 강제)

벤더 (URLLC)	대역폭	capacity_gain
GlobalConnect 5G	400	0.20
TelcoNet Solutions	500	0.25
NextGen Networks	600	0.30

출력 필드	타입	설명
slice_id	string	조달된 슬라이스 식별자. ④ record_decision.slice_id 에 기록
status	string	active · rejected
vendor_id	string	
cost_total	float	위 환산식
capacity_gain	float	①의 add_capacity(amount=...) 에 그대로 넣는다
expires_at_step	int	만료 스텝
reason	string null	거부 사유

```
// 요청
{"vendor_id": "vendor-1", "slice_type": "URLLC",
 "qos_requirements": {"latency": 1.0, "bandwidth": 400, "reliability": 99.99},
 "duration_steps": 10, "current_step": 12}
// 응답
{"slice_id": "slice-urllc-0012-v1", "status": "active", "vendor_id": "vendor-1",
 "cost_total": 625.0, "capacity_gain": 0.250, "expires_at_step": 22, "reason": null}
```

비용은 제약이 아니다. 예산 없음. ④가 procurement_cost_total 을 집계해 논문 지표로 보고한다.

update_rating(vendor_id, outcome)

⑤의 report_outcome 결과를 에이전트가 증계해 호출한다.

파라미터	타입	필수	출처
<code>vendor_id</code>	string	예	⑤ <code>report_outcome().vendor_id</code>
<code>outcome.sla_met</code>	bool	예	⑤ <code>report_outcome().sla_met</code>
<code>outcome.decision_id</code>	string	예	추적용

출력 필드	타입
<code>vendor_id</code>	string
<code>rating_before</code> · <code>rating_after</code>	float
<code>delta</code>	float

갱신식: `rating` $\leftarrow$ `clip(rating +  $\delta$ , 1.0, 5.0)`, `$\delta$` = `+0.05` (충족) / `-0.20` (위반). `vendors.json` 에 반영.

```
// 요청
{"vendor_id": "vendor-1", "outcome": {"sla_met": false, "decision_id": "exp-proposed-emergency-s0-0012"}}
// 응답
{"vendor_id": "vendor-1", "rating_before": 4.80, "rating_after": 4.60, "delta": -0.20}
```

⑤ `slice-feedback` — 계약

담당 B · `claude/spec/feedback.md`

결정을 채점하고 정책별 경험적 신뢰도를 유지한다. 설계 근거는 `rationale/feedback.md`.

`report_outcome(decision_id, observed)`

파라미터	타입	필수	출처
<code>decision_id</code>	string	예	④가 발급한 것
<code>observed</code>	Observation	예	<code>step()</code> 이후의 관측 (= <code>obs_{t+1}</code>)

반드시 `step()` 이후에 호출한다. `obs_{t+1}.allocation` 이 t에 적용된 배분, `obs_{t+1}.traffic` 이 그 배분이 감당해야 했던 수요다. `step()` 전 호출은 `premature_scoring` 오류 (`flow/errors.md`).

출력: Outcome

필드	타입	설명
<code>sla_met</code>	bool	<code>violations</code> 가 전부 <code>false</code>
<code>policy</code>	PolicyName	채점 대상 정책
<code>error</code>	float	0~1. 이상 배분과의 거리
<code>ideal_allocation</code>	SliceTriple	위반이 딱 없어지는 배분
<code>applied_allocation</code>	SliceTriple	실제 적용된 배분 (= <code>observed.allocation</code>)
<code>requested_allocation</code>	SliceTriple	에이전트가 요청했던 배분 (<code>decisions.json</code> 에서 조회)
<code>actuator_delta</code>	float	둘 사이의 거리. 액추에이터가 얼마나 꺾었나
<code>observed_violations</code>	SliceFlags	채점 시점의 위반. MTTR 재구성용
<code>vendor_id</code>	string null	있으면 에이전트가 ③ <code>update_rating</code> 에 증계
<code>reliability_before</code> · <code>reliability_after</code>	float	

error 계산 — 정답을 쓰지 않고 관측만으로 나온다. `capacity` 를 빼면 안 된다.

```
a*    = normalize(traffic / (thresholds × capacity))  # 이상 배분
error = L1(applied_allocation, a*) / 2                # [0, 1]
```

신뢰도 갱신: `r ← (1-0.2)·r + 0.2·(sla_met ? 1 : 0)`

부작용: ④의 해당 레코드에 `outcome` 을 덧붙인다 (`sla_met`, `error`, `scored_at_step`, `applied_allocation`, `requested_allocation`, `actuator_delta`, `observed_violations`).

```
// 요청 - step() 후의 관측
{"decision_id": "exp-proposed-emergency-s0-0012",
 "observed": {"step": 13,
  "traffic": {"embb": 0.571, "urllc": 0.595, "mmtc": 0.390},
  "allocation": {"embb": 0.340, "urllc": 0.490, "mmtc": 0.170},
  "capacity": {"embb": 1.600, "urllc": 1.600, "mmtc": 1.600},
  "utilization": {"embb": 1.050, "urllc": 0.760, "mmtc": 1.435},
  "violations": {"embb": true, "urllc": false, "mmtc": true}, ...}}
// 응답
{"sla_met": false,
 "policy": "rule_based",
 "error": 0.184,
 "ideal_allocation": {"embb": 0.392, "urllc": 0.306, "mmtc": 0.301},
 "applied_allocation": {"embb": 0.340, "urllc": 0.490, "mmtc": 0.170},
 "requested_allocation": {"embb": 0.200, "urllc": 0.700, "mmtc": 0.100},
 "actuator_delta": 0.210,
 "observed_violations": {"embb": true, "urllc": false, "mmtc": true},
 "vendor_id": null,
 "reliability_before": 0.880,
 "reliability_after": 0.704}
```

계산 확인: $a^* = \text{normalize}([0.571/(0.9 \times 1.6), 0.595/(1.2 \times 1.6), 0.390/(0.8 \times 1.6)]) = [0.392, 0.306, 0.301]$, $\text{error} = (|0.340 - 0.392| + |0.490 - 0.306| + |0.170 - 0.301|) / 2 = 0.184$, $r = 0.8 \times 0.880 = 0.704$.

이 예시가 `rule_based` 의 한계를 보여준다. `situation="emergency"` 의 고정 목표 `[0.2, 0.7, 0.1]` 은 과잉 반응이었다 — URLLC에 0.49를 줬는데 이용률 0.76으로 여유가 남았고, 굵긴 mMTC가 1.435로 터졌다. 상황 판단은 맞았는데 정책이 틀렸다. ⑤가 `rule_based` 의 신뢰도를 낮춘다.

get_reliability_table()

입력: 없음 · 출력: `ReliabilityTable`

필드	타입	설명
<code>{policy}.reliability</code>	float	EMA 원값
<code>{policy}.n</code>	int	누적 표본 수
<code>{policy}.effective</code>	float	축소 보정값. 에이전트는 이걸 쓴다 → ④ <code>confidence.empirical</code>
<code>{policy}.recent_error</code>	float	최근 5회 <code>error</code> 의 EMA. ② <code>propose_allocation.recent_error</code> 에 중계

축소 보정: $\text{effective} = (r \cdot n + 0.5 \cdot 5) / (n + 5)$

```
{"rule_based": {"reliability": 0.910, "n": 120, "effective": 0.894, "recent_error": 0.094},
"lstm_forecast": {"reliability": 0.840, "n": 95, "effective": 0.823, "recent_error": 0.112},
"dqn": {"reliability": 0.420, "n": 6, "effective": 0.456, "recent_error": 0.380}}
```

두 종류의 신뢰도 — 종합은 에이전트만 한다.

	제공	의미
내재적 (<code>confidence</code>)	②	이번 입력에 대한 확신
경험적 (<code>effective</code>)	⑤	이 정책이 평소 얼마나 맞았나

```
combined = (situation × intrinsic × effective)^(1/3)
escalate if combined < 0.45
```

셋의 기하평균이다. `situation` 은 에이전트가 내는 상황 판단 자체에 대한 확신이며 ②·⑤ 어느 쪽도 주지 않는다. 이게 없으면 상황을 잘못 읽고도 정책에 자신 있을 때 `combined` 가 높게 나와 에스컬레이션이 일어나지 않는다. `normal` 인지 정확도가 50.5%라 오탐이 구조적으로 많다 (`rationale/environment.md`).

정정 A~K — 코드 대조 검증

담당 B · claude/rationale/corrections.md

기존 코드가 설계 전제와 어긋나는 것 11건

1. 코드 대조 검증 — 정정 사항

분리 설계서 §5의 정정 A-B-C에 이어지는 번호를 쓴다.

정정 D — ④의 `get_metrics()` 가 에이전트에게 정답을 누출한다 ⚠️ 최우선

분리 설계서 ①은 `is_emergency` 를 노출하지 않는다고 못 박았다. 그런데 ④의 `get_metrics()` 반환값에 다음 두 필드가 있다.

```
"perception_accuracy": float,      # 정답 대비 적중/오탐/미탐
"escalation_precision": float,     # 사후 판정
```

두 지표 모두 정답 없이는 계산할 수 없다. ④가 이걸 계산하려면 ④가 정답을 읽어야 하고, ④의 반환값은 MCP 도구 결과로 에이전트 컨텍스트에 그대로 들어간다.

에이전트: `get_metrics()` 호출

- `"perception_accuracy: 0.62"` 수신
- "내가 지금까지 상황 판단을 자주 틀렸구나"
- 이후 판단을 보정
- 정답이 우회 경로로 에이전트에 도달함

①에서 막은 것을 ④가 열어주는 구조다. 심사에서 지적당하면 실험 전체가 무효가 된다.

해소안 — 지표를 두 계층으로 분리한다.

계층	산출 주체	지표	에이전트 접근
관측 가능 지표	④ <code>get_metrics()</code>	<code>steps</code> , <code>interventions</code> , <code>autonomous_rate</code> , <code>sla_violations</code> , <code>mean_utilization</code> , <code>policy_usage</code> , <code>mttr</code>	허용
정답 의존 지표	<code>eval/score.py</code> (오프라인)	<code>perception_accuracy</code> , <code>escalation_precision</code>	차단

`mttr` (복구 시간)은 이용률이 임계값을 벗어난 구간 길이로 정의되므로 관측 가능하다. 정답이 필요 없다.

정답은 ①이 매 스텝 `runs/{run_id}/truth.jsonl` 에 append 하고, 어떤 MCP 도구도 이 파일을 읽지 않는다. 실험 종료 후 `eval/score.py` 가 `truth.jsonl` 과 `decisions.json` 을 `step` 키로 조인해 정답 의존 지표를 계산한다.

정정 E — `rule_based` 는 "수정 없음"이 아니다. 정답을 직접 읽는다

분리 설계서 §3 ②의 출처 코드 표에 `rule_based` → 수정 없음으로 적혀 있다. 실제 코드는 다음과 같다.

```
# ml_orchestrator_demo.py:422
def update_allocation_rule_based(self):
    if self.is_emergency:                # ← :429
        target_allocation = np.array([0.2, 0.7, 0.1])
    elif self.is_special_event:          # ← :433
        target_allocation = np.array([0.6, 0.3, 0.1])
    elif self.is_iot_surge:              # ← :436
        target_allocation = np.array([0.3, 0.3, 0.4])
    else:
        target_allocation = np.array([0.4, 0.4, 0.2])
    ...
```

`rule_based` 는 사람이 CLI로 넣어준 상황 판단을 입력으로 받는 함수다. 이걸 그대로 ②에 옮기면 ②가 정답을 들고 있게 되어 정정 D와 같은 누출이 된다. ②는 무상태이므로 `self.is_emergency` 를 가질 수도 없다.

해소안 — 상황 라벨을 파라미터로 승격한다. 이것이 오히려 본 연구의 기여를 가장 깨끗하게 보여주는 지점이다.

```
propose_allocation(
    policy = "rule_based",
    observation = {...},
    situation = "emergency" | "special_event" | "iot_surge" | "normal", # ← 에이전트가 추론해 넣음
    history = None
)
```

	베이스라인	제안 기법
<code>situation</code> 값의 출처	CLI <code>--emergency</code> (사람)	에이전트 추론
호출되는 함수	<code>update_allocation_rule_based()</code>	동일
배분 로직	동일	동일

정확히 한 변수만 바꾼다. 로직이 같으므로 성능 차이는 전부 "상황 인지가 맞았는가"에서 나온다. 비교군 1(상황 인지만 에이전트)이 통제 실험으로 성립한다.

`situation` 파라미터는 필수로 둔다. 기본값을 `"normal"` 로 주면 에이전트가 생략했을 때 조용히 평시로 처리되어 미탐이 측정에서 사라진다.

정정 F — 시뮬레이터가 벽시계를 읽는다. 재현이 불가능하다

```
# ml_orchestrator_demo.py:297 (generate_traffic)
hour_of_day = datetime.now().hour / 24.0
day_of_week = datetime.now().weekday() / 6.0

# ml_orchestrator_demo.py:500 (create_feature_vector)
hour_of_day = datetime.now().hour / 24.0
```

세 가지 문제가 동시에 발생한다.

1. **재현 불가** — 같은 시나리오를 오후 2시와 새벽 3시에 돌리면 트래픽이 다르다. 비교군 4개를 각각 다른 시각에 돌리면 공정한 비교가 아니다.
2. **관측의 context.hour_of_day 가 가짜** — 에이전트는 "지금 새벽이니 트래픽이 낮을 것"이라 추론하는데, 100스텝을 1초에 돌리면 시간은 사실상 멈춰 있다.
3. **reset(seed=) 무력화** — `np.random.normal (:310, :512)`이 전역 RNG를 쓴다. 시드를 고정해도 벽시계 향이 남는다.

해소안 — ①에 가상 시계와 전용 RNG를 넣는다.

```
class SliceEnv:
    def __init__(self, seed=0, minutes_per_step=15, start_hour=8):
        self.rng = np.random.default_rng(seed) # 전역 RNG 사용 금지
        self.step_idx = 0
        self._mps, self._h0 = minutes_per_step, start_hour

    @property
    def sim_time(self):
        total_min = self._h0 * 60 + self.step_idx * self._mps
        return {"hour_of_day": (total_min // 60) % 24,
                "day_of_week": (total_min // 1440) % 7}
```

`datetime.now()` 3곳과 `np.random` 2곳을 위 두 개로 치환한다. 수정량 5줄, 효과는 실험 재현성 전체.

`minutes_per_step=15` 이면 96스텝 = 하루. `mixed` 시나리오(60스텝)가 반나절 이상을 덮어 일간 주기가 관측에 실제로 나타난다.

정정 G — 벤더 데이터 모델이 두 벌이고 서로 호환되지 않는다

분리 설계서 §3 ③의 도구 시그니처는 두 모델을 섞어 쓰고 있다.

	모델 A	모델 B
파일	5G-Marketplace/data/vendors.json	5G-Marketplace/data/vendor_registry/*.json
구조	리스트 5개, offerings 가 slice_type으로 중첩	dict, offering_id 로 평탄화 (6건 / 벤더 3곳)
위치 필드	location: "New York, USA"	locations: ["us-east", "us-west"]
rating	있음 (4.5 ~ 4.9)	없음
가격	cost	price_per_hour
소비 함수	score_vendor_offering() engine.py:719	_advanced_score_offer() engine.py:185

분리 설계서의 list_offerings(slice_type, region) 은 모델 B의 시그니처인데, rating 을 반환하고 engine.py:792 를 근거로 든다. engine.py:792 (vendor.get("rating", 3.0))는 모델 A만 읽는다. 모델 B에는 rating 이 없으므로 항상 기본값 3.0으로 떨어지고, ⑤의 레이팅 갱신이 점수에 전혀 반영되지 않는다.

해소안 — 모델 A로 단일화하고 모델 B와 그 경로를 버린다.

- ③은 score_vendor_offering(vendor, slice_type, qos) 만 쓴다. 모델 A의 shape와 정확히 일치하고 rating 을 읽는다.
- 부트스트랩 시 모델 A를 data/vendors.json 으로 복사하면서 regions 필드만 추가한다(벤더 5곳 × 1줄).
- query_vendors() / find_matching_offerings() / _advanced_score_offer() 는 사용하지 않는다.

부수 효과: 분리 설계서가 지정한 engine.py:115 의 await 누락(→ query_vendors() 가 항상 [] 반환)이 고칠 필요 없이 사라진다. 해당 경로를 안 쓰기 때문이다. ③의 작업량이 한 단계 더 내려간다.

정정 H — ②의 LSTM 경로에 조용한 폴백이 박혀 있다

```
# ml_orchestrator_demo.py:341 update_allocation_ml
try:
    if len(self.feature_history) >= self.sequence_length:
        ...
    else:
        logger.info("Not enough history...")
        return self.update_allocation_rule_based() # ← :414 조용한 폴백
except Exception as e:
    logger.error(f"Error in ML allocation: {e}")
    return self.update_allocation_rule_based() # ← :419 조용한 폴백
```

분리 설계서 §1의 개입 지점 5번이 "이상 시 원인 분석 — 조용한 폴백으로 로그 수동 추적"이다. 이 코드가 바로 그 조용한 폴백이다.

이 상태로 ②에 이식하면 propose_allocation(policy="lstm_forecast") 가 실제로는 rule_based 결과를 반환하면서 "policy": "lstm_forecast" 라고 답한다. ④의 policy_usage 집계가 통째로 거짓이 되고, ⑤의 정책 신뢰도는 lstm의 성적으로 rule의 성적을 기록한다. 자기 개선 루프가 잘못된 대상을 학습한다.

해소안 — ②는 폴백하지 않는다. 실패를 실패로 반환한다.

```
propose_allocation(policy="lstm_forecast", ...) -> {
  "policy": "lstm_forecast",
  "allocation": None,
  "confidence": 0.0,
  "status": "unavailable",
  "reason": "history_insufficient: 4 < 10"
}
```

폴백 여부는 **에이전트가 결정한다**. 그것이 곧 "정책 선택"이고, 이 연구의 주제다. 대체 정책을 고르는 것 자체가 ④에 별도 결정으로 기록되어야 한다.

정정 I — TensorFlow가 의존성에 아예 없고, 모델은 SavedModel 형식이다

```
requirements.txt → tensorflow 항목 없음 (numpy, torch, fastapi, ... 만 존재)
현재 환경       → Python 3.12.0 / tensorflow 미설치
```

그런데 lstm_predictor.py:20 과 dqn_classifier.py:15 는 `from tensorflow.keras.models import load_model` 을 최상단에서 import 한다. 분리 설계서 §7의 0단계가 실패하면 ②의 절반이 날아간다. 이건 단순 설치 문제가 아니라 형식 문제도 겸한다.

모델	경로	형식
SliceAllocationPredictor	5G-Network-Slicing/models/lstm_model_20250601_011235/	SavedModel (saved_model.pb + variables/)
TrafficClassifier	5G-Network-Slicing/models/dqn_model_20250601_012010/	SavedModel
AutoregressiveLSTMPredictor	5G-Network-Slicing/models/enhanced_lstm/	.h5
단일 LSTM	models/lstm_single/best_model.h5	.h5 (+ X_scaler.npy)

핵심 위험: TF 2.16부터 Keras 3가 기본이 되며, Keras 3의 `load_model()` 은 SavedModel 디렉토리를 직접 적재하지 못한다. TFSMLayer 로 감싸거나 `tf.saved_model.load()` 를 써야 하고, 후자는 .predict() 인터페이스를 주지 않는다. 즉 `pip install tensorflow` 만 하면 최신 버전이 깔려서 두 모델 다 실패한다.

해소안 — 버전을 고정한다.

```
Python 3.10 + tensorflow==2.15.1 # Keras 2. load_model()이 SavedModel을 그대로 읽음
```

분리 설계서 §8의 "Python 3.10"은 맞다. 여기에 TF 버전 상한이 빠져 있었다. `tensorflow>=2.x` 로 적으면 0단계에서 바로 막힌다.

추가 미확인 항목 — 스케일러: `lstm_model_20250601_011235/`에는 스케일러 파일이 없다

(`models/lstm_single/`에만 `x_scaler.npy` 존재). 학습 시 정규화를 했는지, 했다면 어떤 스케일인지 불명이다. 0단계 검증 스크립트가 적재 성공만이 아니라 출력이 그럴듯한지(합 ≈ 1 , 각 항 0~1)까지 확인해야 한다. 적재는 되는데 출력이 `[-3.2, 8.1, 0.4]` 같은 값이면 스케일러가 필요하다는 뜻이고, 없으면 재학습이다.

정정 J — `explain_score`와 `score_offerings`의 총점이 어긋난다

도구 명세(`T00LS.md`) 작성 중 두 함수를 실행해 확인했다. 같은 벤더 · 같은 QoS인데 결과가 다르다.

함수	결과
<code>score_vendor_offering()</code> :719	99.20
<code>get_score_breakdown()</code> :819	92.00

레이팅을 읽는 키가 서로 다르기 때문이다.

```
# engine.py:792 (score_offerings 경로)
reputation_score = vendor.get("rating", 3.0) / 5.0          # 4.8 / 5.0 = 0.96

# engine.py:891 (get_score_breakdown 내부)
reputation_score = offer.get("reputation_score", 3.0) / 5.0 # 키 없음 → 3.0 / 5.0 = 0.60
```

`vendors.json`에는 `rating`만 있고 `reputation_score`는 없다. URLLC의 평판 가중치 $0.2 \times (4.8 - 3.0) / 5 \times 100 = 7.2$ 점 — 관측된 차이와 정확히 일치한다.

그대로 노출하면 에이전트가 모순된 두 숫자를 받고, 근거 기록(④)에도 잘못된 값이 남는다.

→ `explain_score`는 `score_vendor_offering`의 계산을 분해해 새로 구현한다(약 30줄).

`get_score_breakdown()`을 호출하지 않는다. 반환값의 `neural_network` 키(601바이트, UI용 장식)도 버린다.

정정 K — 조달이 환경에 아무 영향을 주지 못한다

원 모델은 `utilization = traffic / allocation`이고 `allocation`은 비율이라 합이 항상 1.0이다. 코드에 `capacity / total_resource` 계열 변수가 하나도 없다(확인함). 따라서 `procure()`를 호출해도 분모가 커지지 않는다.

- 조달해도 이용률이 안 내려간다 → SLA가 개선되지 않는다
- SLA 결과가 벤더와 무관하다 → `update_rating`의 δ 가 의미 없는 신호가 된다
- ⑤ → ③ 피드백 루프 전체가 공회전한다

설계서 ③의 역할은 "내부 재배분으로 부족할 때 외부에서 조달"인데, 재배분은 파이를 다시 자를 뿐 총량을 늘리지 못한다.

결정 — ①에 슬라이스별 용량 배수를 도입한다.

```
utilization[i] = traffic[i] / (allocation[i] × capacity[i])
capacity_gain = (bandwidth / REFERENCE_BANDWIDTH[slice_type]) × GAIN_SCALE
```

상수	값	근거
REFERENCE_BANDWIDTH	{eMBB: 1100, URLLC: 500, mMTC: 120}	vendors.json 슬라이스별 중앙값 실측
GAIN_SCALE	0.25	조달 1회 = 압력 20% 감소, 완전 해소는 불가
CAPACITY_BASE	1.6	기본 용량. \$1.7
CAPACITY_MAX	2.6	슬라이스당 상한. \$1.7

파생 지표 $demand_pressure = \sum_i traffic / (\theta \times capacity)$ 를 Observation 에 노출한다. 1.0을 넘으면 어떤 배분으로도 전 슬라이스 SLA를 지킬 수 없다 — 재배분과 조달의 경계를 가르는 유일한 근거이며, 이게 없으면 에이전트는 모든 값이 비율이라 얼마나 부족한지 절대량을 알 수 없다.

상세는 TOOLS.md \$2.1 · \$2.6 · \$4.5.

정정 요약

#	내용	영향	조치 시점
D	④ get_metrics() 가 정답 누출	실험 유효성	④ 구현 전 (필수)
E	rule_based 가 상황 플래그를 직접 읽음	통제 실험 성립 여부	② 구현 전 (필수)
F	벽시계·전역 RNG 사용	재현성	① 구현 시
G	벤더 데이터 모델 두 벌	⑤→③ 피드백 단절	③ 구현 전
H	②의 조용한 폴백	④⑤ 집계 오염	② 구현 시
I	TF 미설치 + SavedModel 형식	0단계 성패	최우선
J	explain_score 총점 7.2점 불일치	에이전트가 모순된 값 수신	③ 구현 시
K	조달이 환경에 영향 없음	③⑤ 피드백 루프 공회전	① 구현 전 (필수)

D · E · K는 논문 주장의 근거를 직접 무너뜨린다. 나머지는 고치지 않으면 결과가 지저분해지는 수준이다.

부록 — 1단계 분리 설계서 대비 변경점

원본은 origin/decomposition.md .

분리 설계서	이 문서	사유
④ <code>get_metrics()</code> 에 <code>perception_accuracy</code> 포함	제외. <code>eval/score.py</code> 로 이관	정답 누출 (정정 D)
<code>propose_allocation(policy, observation, history)</code>	<code>situation</code> 인자 추가 (필수)	<code>rule_based</code> 가 상황 플래그 의존 (정정 E)
① <code>get_observation()</code> 에 <code>context</code>	<code>sim_time</code> . 가상 시계 기반	벽시계 사용 (정정 F)
③ <code>list_offerings(..., region) + rating</code>	모델 A 단일화. registry 경로 폐기	두 모델 비호환 (정정 G)
② 실패 시 동작 미정	<code>status</code> 필드. 폴백 금지	조용한 폴백 (정정 H)
§8 "Python 3.10"	+ <code>tensorflow==2.15.1</code> 고정	SavedModel × Keras 3 (정정 I)
<code>procure(duration_hours)</code>	<code>duration_steps</code>	가상 시계와 단위 통일
평활·클립이 정책 내부	①의 <code>apply_allocation()</code> 으로 이관	정책 비교 공정성 (§3.2)
<code>engine.py:115</code> <code>await</code> 수정 필요	수정 불필요	해당 경로 미사용 (정정 G)
<code>lstm_forecast</code> 작업량 小~中	0단계 3번 결과에 종속	스케일러 부재 (정정 I)

계약 검증 — 값 유실 20건

담당 B · `claude/rationale/verification.md`

생산자 없는 소비 · 소비자 없는 생산 · 근본 원인 3패턴 · 재발 방지

1.5 도구 계약 검증 — 값 유실 9건

정정 A~K가 기존 코드의 문제였다면, 이 절은 `TOOLS.md`의 도구 계약 자체를 검증한 결과다. 도구 21개의 모든 입력 필드에 대해 "이 값을 누가 생산하는가"를, 모든 출력 필드에 대해 "이 값을 누가 소비하는가"를 역추적했다.

생산자 없는 소비 4건, 소비자 없는 생산 5건이 나왔다. 전부 코드 버그가 아니라 계약의 구멍이며, 구현 전에 메우지 않으면 통합 시점에 드러난다.

#	값	증상	등급	담당
V2	<code>decision_id</code> (에스컬레이션 시)	그 스텝의 SLA 결과가 통계에서 소멸	치명	A
V3	<code>run_id</code>	<code>truth.jsonl</code> × <code>decisions.json</code> 조인 불가	치명	A · C
V4	<code>recent_error</code>	②의 LSTM 신뢰도를 계산할 수 없음	중대	B · C
V5	<code>applied_allocation</code>	기록(요청)과 채점(적용)의 기준 불일치	중대	B
V6	<code>outcome.sla_met</code>	<code>sla_violations</code> 가 1스텝 어긋남	중대	A
V7	<code>in_distribution</code>	논문 주장을 하되 검증 불가	경미	A · B
V8	<code>demand_class</code>	분류기 오류와 판단 오류를 구분 불가	경미	A · C
V9	<code>situation</code> ↔ <code>truth</code> 매핑	채점 기준이 사후 해석 가능	경미	A
V10	$\sum \text{policy_usage}$ vs $\sum n$	정상 차이를 버그로 오인	경미	A

도구별 반영 내용은 `TOOLS.md`의 해당 절에 ⚠️ 블록으로 들어가 있다. 여기서는 왜 생겼는가와 어떻게 막는가만 적는다.

근본 원인 — 세 가지 패턴

아홉 건이 무작위로 생긴 게 아니다. 서버를 나눈 방식에서 구조적으로 유도된다.

패턴 1 — 호출 순서가 값의 가용 시점을 어긋나게 한다 (V5, V6)

`record_decision` (3단계)이 `apply_allocation` (4단계)보다 먼저다. 기록 시점에 "실제 적용된 값"은 아직 존재하지 않는다. 그런데 채점은 적용값 기준이다.

기록 시점 **t**: 요청 `allocation = {0.20, 0.70, 0.10}` ← ④에 남는 것
적용 시점 **t**: 실제 `allocation = {0.34, 0.49, 0.17}` ← 아무데도 안 남던 것
채점 시점 **t+1**: `error`는 적용값 기준 = `0.086`
요청값 기준이었다면 = `0.316` ← 4배 차이

같은 이유로 ④가 보는 `observation.violations` 는 직전 결정의 결과라 1스텝 어긋난다.

판정법: 도구 X의 출력이 도구 Y의 입력인데 호출 순서가 $Y \rightarrow X$ 라면, 그 값은 유실된다. 순서를 바꾸거나, 나중에 그 값을 보는 제3자(여기서는 ⑤)가 채워 넣어야 한다.

패턴 2 — 무상태 원칙이 누적값의 공급선을 끊는다 (V4)

②는 무상태이고 ⑤를 호출할 수 없다(서버 간 직접 호출 금지, §2.2). 그런데 `lstm_forecast` 의 신뢰도 `exp(-3 \cdot \bar{e})` 에서 $\bar{e}$ 는 누적 오차다. ②가 원리적으로 알 수 없는 값을 식에 넣어 두었다.

해결은 §2.2의 원칙 그대로다 — 에이전트가 중계한다. ⑤가 `recent_error` 를 반환하고, 에이전트가 `propose_allocation` 의 인자로 넘긴다. ②는 여전히 무상태다.

판정법: 무상태 서버의 산출식에 "최근 N회", "누적", "평균" 같은 말이 들어가면 그 값은 반드시 인자여야 한다.

패턴 3 — 식별자를 쓰기만 하고 발급하지 않는다 (V2, V3)

`decision_id = {run_id}-{step:04d}` 라고 형식만 정하고 `run_id` 를 발급하는 도구를 두지 않았다. 에스컬레이션도 마찬가지로 `escalation_id` 만 주고 `decision_id` 를 안 줘서, 그 스텝은 `report_outcome` 을 호출할 방법이 없었다.

판정법: 명세에 등장하는 모든 식별자에 대해 "이걸 처음 만드는 도구가 무엇인가" 를 답할 수 있어야 한다. 답이 없으면 유실이다.

V2가 가장 위험한 이유

나머지 여덟 건은 지표가 틀리거나 분석이 제약되는 수준이다. V2만 논증 자체를 무너뜨린다.

에스컬레이션한 스텝은 정의상 에이전트가 가장 자신 없던 스텝이다. `decision_id` 가 없어 채점되지 않으면 그 스텝들이 통계에서 통째로 빠진다.

proposed arm: 어려운 스텝 4개를 에스컬레이션 → 그 4개가 집계에서 제외
baseline arm: 에스컬레이션 없음 → 60개 전부 집계

비교군마다 표본이 다르다. `proposed` 의 `sla_violations` 가 낮게 나와도 그건 어려운 스텝을 빼고 센 결과다. 핵심 논증인 "개입은 줄었는데 SLA 위반은 늘지 않았다" 가 편향된 표본 위에서 성립한다. 심사에서 지적당하면 방어할 말이 없다.

→ `record_escalation` 이 `escalation_id` 와 `decision_id` 를 한 번의 호출로 둘 다 발급한다. 나눠 두면 에이전트가 빠뜨릴 수 있다.

재발 방지 — 계약 검사

도구는 앞으로도 늘어난다(이미 20 → 21). 사람이 매번 역추적하면 또 놓친다.

`docs/contract.json`에 기계가 읽을 수 있는 계약을 둔다. 도구마다 필수 입력, 출력 필드, 그리고 각 필수 입력의 생산자를 명시한다.

```
{
  "propose_allocation": {
    "server": "policy",
    "requires": {
      "observation": "observe.get_observation",
      "situation": "AGENT",
      "recent_error": "feedback.get_reliability_table"
    },
    "produces": ["policy", "allocation", "confidence", "in_distribution", "status"]
  }
}
```

`tools/check_contract.py`가 검사할 것:

1. 모든 필수 입력에 생산자가 있는가 (AGENT 명시 포함) → V3, V4 유형 차단
2. 생산자의 호출 순서가 소비자보다 앞인가 → V5, V6 유형 차단
3. 명세에 등장하는 모든 식별자에 발급 도구가 있는가 → V2 유형 차단
4. `produces` 중 아무도 소비하지 않는 필드 목록 → V7, V8 유형 경고 (의도적일 수 있으므로 경고)

AGENT가 생산자인 값은 네 개뿐이고, 그 넷이 연구의 기여 지점 전체다 — `situation`, `confidence.combined`, 에스컬레이션 판단, 정책 선택. 이 목록이 늘어나면 에이전트가 해야 할 판단이 서버로 새고 있다는 뜻이므로, 검사기가 넷을 넘으면 경고하도록 한다.

소유는 C다. 목 서버와 같은 성격 — 계약의 실행 가능한 명세이고, C는 A-B의 서버를 소비하는 쪽이라 불일치를 가장 먼저 만난다.

1.6 전면 재검증 — 추가 11건

§1.5 이후 `capacity` 도입(정정 K)으로 계약이 바뀌었다. 세 문서를 교차 대조하고 모든 예시를 재계산한 결과 11건이 더 나왔다. 절반은 `capacity` 도입이 만든 파급이고, 절반은 처음부터 있던 사각지대다.

#	항목	증상	등급	담당
W1	조달이 루프의 어디에 들어가는가	순서 미정. <code>add_capacity</code> 가 <code>step()</code> 뒤로 가면 조달한 스텝이 효과를 못 봄	중대	C
W2	③이 현재 스텝을 모른다	<code>expires_at_step</code> 을 계산할 수 없음	중대	B
W3	<code>cost_total</code> 단위 불일치	시간당 단가 × 스텝 수 → 비용 4배 부풀림	중대	B
W4	⑤가 <code>vendor_id</code> 를 모른다	<code>update_rating</code> 중계 불가	중대	A · B
W5	실행 생명주기 미정	60회 산출물이 한 파일에 덮어쓰기	치명	A · C
W6	<code>compare_policies</code> 의 <code>recent_error</code>	스칼라라 전 정책이 같은 오차 공유	중대	B
W7	① 도구 목록에 <code>add_capacity</code> 누락	설계서 ↔ 명세서 불일치	경미	—
W8	LSTM이 <code>capacity</code> 를 못 봄	11차원 고정. 한계로 명시 필요	경미	B
W9	<code>demand_pressure</code> 계산 오류	1.061 → 1.069	경미	—
W10	컨텍스트 예산이 20개 기준	21개 + <code>Observation</code> 확장 반영	경미	C
W11	오류 규약에 조달 케이스 없음	상한 초과·순서 위반 미정의	경미	A · B

W5가 가장 위험하다

§1.5의 V3(`run_id` 부재)를 고치고도 절반만 해결했다. `run_id` 를 발급하게 만들었지만, 산출물 경로는 여전히 `data/decisions.json` 단일이었다.

```
exp-baseline-normal-s0    → data/decisions.json (덮어씀)
exp-arm1-normal-s0       → data/decisions.json (덮어씀)
...60회...
                          → 마지막 1회분만 남음
```

`reset()` 이 ①만 초기화한다는 점도 명시되지 않았다. ③의 벤더 레이팅과 ⑤의 신뢰도는 실행을 넘어 살아남는데, 그게 의도인지 버그인지 문서 어디에도 없었다. `cold / warm` 모드는 §6.2에 있었으나 파일 경로와 연결되지 않아 실행 불가능한 규약이었다.

→ §5.0에 실행 생명주기를 명시했다. `runs/{run_id}/` 로 분리하고, 서버별로 무엇이 초기화되고 무엇이 남는지 표로 고정했다.

W1·W2·W3 — capacity 도입의 파급

정정 K는 ①에 용량을 넣는 것으로 끝나지 않았다. 조달이 실제로 동작하게 되자 그동안 무해했던 공백이 드러났다.

- **W1:** 조달이 장식일 때는 `procure()` 를 언제 부르든 상관없었다. 이제 `add_capacity` 가 `step()` 뒤로 가면 조달한 스텝이 효과를 못 본다 — 에이전트 입장에서 "샀는데 아무 일도 안 일어났다" 가 되어 조달 학습이 왜곡된다.
- **W2:** 만료가 없을 때는 `expires_at_step` 이 의미 없었다. 이제 ③이 그 값을 계산해야 하는데 ①과 별개 프로세스라 현재 스텝을 모른다. 에이전트가 `current_step` 을 넘겨야 한다.
- **W3:** 비용이 집계되지 않을 때는 단위가 무의미했다. 이제 `procurement_cost_total` 이 논문 지표이므로 시간당 단가 × 스텝 수를 그대로 곱하면 4배 부풀려진다.

```
cost_total = cost × duration_steps × (MINUTES_PER_STEP / 60)
            = 250 × 10 × 0.25 = 625.0          # 2500.0 이 아니다
```

교훈: 죽은 기능은 계약의 구멍을 감춘다. 정정 K로 ③이 살아나자 W1·W2·W3가 동시에 드러났다. 1차 범위에서 제외한 `dqn` 도 2차에 넣는 순간 같은 일이 일어날 것이다.

W6 — V4를 고치면서 만든 새 구멍

§1.5의 V4를 고쳐 `propose_allocation` 에 `recent_error` (스칼라)를 넣었다. 그런데 `compare_policies` 는 정책을 동시에 평가한다. 스칼라를 받으면 모든 정책이 같은 오차 이력을 공유하는 셈이라 신뢰도 비교가 무의미해진다.

→ `compare_policies` 는 `recent_errors: {policy: float}` 딕셔너리를 받는다. ⑤의 `get_reliability_table()` 출력을 그대로 넘기면 된다.

수정이 새 구멍을 만든다. W6은 V4의 직접적 부산물이다. §1.5의 계약 검사기(`check_contract.py`)를 수정할 때마다 돌려야 하는 이유다.

② `slice-policy` — 설계 근거

담당 B · [claude/rationale/policy.md](#)

계약은 `spec/policy.md`.

`list_policies().available` 이 핵심인 이유

TensorFlow 적재에 실패하면 서버는 죽지 않고 해당 정책만 `false` 로 내린다. 에이전트가 "왜 이걸 못 쓰나"를 알 수 있어야 대체 정책을 고를 수 있다.

`description` 은 프롬프트다

MCP가 이 문자열을 LLM 컨텍스트에 그대로 넣는다. "평시 효율 우수" 같은 문구는 사실상 "언제 이걸 골라라"는 조언이다. 계약의 예시는 `minimal` 판이며, `advisory` 판과의 비교가 실험 변수다 (MCP-design.md §6.3).

`situation` 에 기본값을 두지 않는 이유

"normal" 을 기본값으로 주면 에이전트가 생략했을 때 조용히 평시로 처리되어 **미탐이 측정에서 사라진다**.

`situation` 은 `rule_based` 에서만 배분에 영향을 준다. 그래도 모든 정책이 필수로 받는 이유: 에이전트가 **매 결정마다 상황 판단을 명시하도록 강제**하고, 그 값이 ④에 기록되어 `perception_accuracy` 의 입력이 되기 때문이다. 이게 없으면 에이전트가 무엇이라 판단했는지 측정할 방법이 없다.

폴백 금지의 근거

원본 `update_allocation_ml()` (:341)은 실패 시 `update_allocation_rule_based()` 를 조용히 호출한다 (:414, :419). 그대로 옮기면 **rule_based 결과를 반환하면서 "policy": "lstm_forecast" 라고 답한다**. ④의 `policy_usage` 가 거짓이 되고 ⑤가 잘못된 대상을 학습한다.

대체 정책 선택은 에이전트의 일이다. 그게 곧 "정책 선택"이고 이 연구의 주제다.

⚠ `recent_error` 중계 경로가 필요한 이유

`lstm_forecast` 의 내재적 신뢰도는 $\exp(-3 \cdot \bar{e})$ 이고, $\bar{e}$ 는 **최근 5회 예측 오차의 EMA**다. 그런데 `error` 를 계산하는 주체는 ⑤이고, ②는 **무상태라 ⑤를 조회할 수 없다** (서버 간 직접 호출 금지).

공급 경로를 명시하지 않으면 여기서 끊긴다.

```
⑤.get_reliability_table() → {"lstm_forecast": {..., "recent_error": 0.112}}
```

에이전트가 중계

```
②.propose_allocation(policy="lstm_forecast", ..., recent_error=0.112)
```

②는 여전히 무상태다. 누적값을 들고 있는 건 ⑤이고, ②는 인자로 받은 값을 식에 넣을 뿐이다.

rule_based confidence 계산 확인

$0.50 + 0.30 \times \min(1, \min_i |u_i - \theta_i| / \theta_i)$ → 여유가 가장 작은 것은 mMTC ($|0.95 - 0.80| / 0.80 = 0.188$) → $0.50 + 0.30 \times 0.188 = 0.556$

이용률이 임계값에 붙어 있을수록 규칙의 이산 분기가 자의적이므로 확신이 낮다.

compare_policies 가 recent_errors 를 딕셔너리로 받는 이유

propose_allocation 은 정책 하나를 다루므로 recent_error 가 스칼라지만, compare_policies 는 전 정책을 동시에 평가하므로 정책마다 다른 값이 필요하다. 스칼라를 받으면 모든 정책이 같은 오차 이력을 공유하는 셈이 되어 신뢰도 비교가 무의미해진다.

실패한 정책도 status 와 함께 돌려줘야 에이전트가 "왜 이걸 못 쓰나"를 알 수 있다.

매 스텝 호출하면 컨텍스트가 빠르게 찬다 (rationale/context-budget.md).

classify_demand 가 상황을 말하지 않는 이유

"지금 어떤 종류의 수요가 지배적인가"만 답한다. URLLC 수요가 지배적인 것과 비상 상황인 것은 다르다 — 야간 저부하에서도 URLLC가 지배적일 수 있다. 상황 판단은 여전히 에이전트의 몫이며, ①의 설계 원칙(정답 비노출)을 위반하지 않는다.

③ `slice-market` — 설계 근거

담당 B · claude/rationale/market.md

계약은 `spec/market.md`.

`qos_requirements` 키에 단위를 붙이지 않는 이유

`engine.py:79` 의 `latency_ms` 는 쓰지 않는 다른 경로(`_rule_based_classification`)의 것이다. 실제 점수 경로(`:339`, `:368`, `:398`)는 `latency · bandwidth · reliability` 만 읽는다. 혼동하면 전부 기본값으로 떨어져 모든 벤더가 같은 점수를 받는다.

`score_offerings` 실측 결과에서 볼 것

1위와 3위가 흥미롭다. `vendor-3`이 모든 QoS 지표에서 우월하고 `rating`도 최고(4.9) 인데 3위다. 가격(300 vs 250)이 URLLC 기준가 250을 초과해 `price_score` 에서 손해를 본다. 에이전트가 "최고 사양"이 아니라 "요구 대비 최적"을 고르는지 관찰할 수 있는 지점이다.

⚠ 정정 J — `get_score_breakdown()` 을 그대로 쓰면 총점이 어긋난다

실행해서 확인한 결과다. 같은 벤더·같은 QoS인데 두 함수가 다른 값을 낸다.

함수	결과
<code>score_vendor_offering()</code> :719	99.20
<code>get_score_breakdown()</code> :819	92.00

원인은 레이팅을 읽는 키가 서로 다르기 때문이다.

```
# engine.py:792 (score_offerings가 쓰는 경로)
reputation_score = vendor.get("rating", 3.0) / 5.0          # → 4.8 / 5.0 = 0.96

# engine.py:891 (get_score_breakdown 내부)
reputation_score = offer.get("reputation_score", 3.0) / 5.0 # → 키 없음 → 3.0 / 5.0 = 0.60
```

`vendors.json` 에는 `rating` 만 있고 `reputation_score` 는 없다. 따라서 `get_score_breakdown` 은 항상 기본값 3.0을 쓴다. URLLC의 평판 가중치 $0.2 \times (4.8 - 3.0) / 5 \times 100 = 7.2\text{점}$ — 관측된 차이와 정확히 일치한다.

그대로 노출하면 에이전트가 모순된 두 숫자를 받는다. `score_offerings` 는 99.2라 하고 `explain_score` 는 92.0이라 한다. 어느 쪽을 믿어야 할지 알 수 없고, 근거 기록(④)에도 잘못된 값이 남는다.

추가로 버릴 것: `get_score_breakdown()` 의 반환값에는 `neural_network` 키(601바이트)가 있다. 점수 계산을 신경망처럼 그려 보이는 **비용** 장식이며 에이전트에게 무의미하다. 매 호출마다 컨텍스트를 먹으므로 반환하지 않는다.

procure 가 duration_hours 가 아니라 duration_steps 인 이유

①이 가상 시계를 쓴다. 단위를 섞으면 만료 시점 계산이 어긋난다.

current_step 이 필요한 이유: ③은 ①과 별개 프로세스이고 시뮬레이션 상태에 접근할 수 없다. 그런데 $\text{expires_at_step} = \text{current_step} + \text{duration_steps}$ 를 반환해야 한다. 에이전트가 observation.step 을 넘겨준다.

비용 환산을 빠뜨리고 두 단위를 그냥 곱하면($250 \times 10 = 2500$) 비용이 4배 부풀어 조달 비용 지표가 무의미해진다.

slice_id 를 ④의 record_decision 에 함께 기록해야 ⑤가 결과 보고 시 vendor_id 를 되찾을 수 있다.

용량 환산 — capacity_gain 이 존재하는 이유

원래 ①의 모델에는 총량 개념이 없어 $\text{utilization} = \text{traffic} / \text{allocation}$ 이었고, allocation 은 비율이라 조달해도 분모가 커지지 않았다. 조달이 SLA에 영향을 주지 못하면 update_rating 의 δ 가 의미 없는 신호가 되고 ⑤ → ③ 피드백 루프 전체가 공회전한다.

결정: ①에 슬라이스별 용량 배수를 도입한다.

```
utilization[i] = traffic[i] / (allocation[i] × capacity[i])
```

capacity 는 {1.0, 1.0, 1.0} 에서 시작하고 조달로만 오른다. 전체 하나가 아니라 슬라이스별인 이유: URLLC를 샀는데 eMBB 이용률이 같이 내려가면 물리적으로 맞지 않는다.

REFERENCE_BANDWIDTH 의 근거

벤더의 대역폭이 단위(Mbps)부터 환경의 추상 트래픽 단위와 다르므로 환산이 필요하다. vendors.json 벤더 5곳의 슬라이스별 중앙값 실측치({eMBB: 1100, URLLC: 500, mMTC: 120})로 나누면 "평균적인 offering = 1.0배" 가 되어 벤더 간 차이만 남는다. URLLC 벤더 3곳의 capacity_gain 이 0.20 / 0.25 / 0.30으로 50% 폭이라 에이전트의 벤더 선택이 결과로 드러난다.

GAIN_SCALE = 0.25 의 튜닝 기준

const.py 에 두고 조정 가능하다. 두 조건을 동시에 만족해야 한다.

1. 조달 1회가 눈에 보일 것 — 해당 슬라이스 압력이 약 20% 내려간다 (1/1.25)
2. 조달만으로 위기가 해소되지 않을 것 — 해소되면 "항상 사는 것"이 지배 전략이 되어 판단이 사라진다

실측 확인: emergency 시나리오의 demand_pressure 1.436을 1.0 아래로 내리려면 GAIN_SCALE=0.25 에서 조달 5회가 필요하다. 상한 2.0에 걸리므로 완전 해소는 불가능하다. 두 조건 모두 만족한다.

비용이 제약이 아닌 이유

상한과 만료 외에 조달을 억제하는 장치가 없으므로, 에이전트는 원리적으로 계속 살 수 있다. 예산을 도입하는 대신 ④가 procurement_cost_total 을 집계해 논문 지표로 보고한다. "개입도 줄고 SLA도 유지했지만 비용이

2배" 라면 그것도 정직한 결과다.

update_rating의 δ 가 비대칭(+0.05 / -0.20)인 이유

초기 레이팅이 4.5~4.9로 좁게 몰려 있어 작은 δ 로는 순위가 바뀌지 않는다. 0.2 하락 = 총점 0.4~0.8점 하락이며, 60~120스텝에서 순위 역전이 관측 가능한 크기다.

에이전트가 중계하는 이유: 서버 간 직접 호출이 없기 때문이다.

⑤ `slice-feedback` — 설계 근거

담당 B · [claude/rationale/feedback.md](#)

계약은 `spec/feedback.md`.

`report_outcome` 이 추가 인자 없이 `observed` 하나만 받는 이유

`obs_{t+1}.allocation` 이 곧 t 에 적용된 배분이고, `obs_{t+1}.traffic` 이 그 배분이 감당해야 했던 수요다. 두 값이 같은 관측 안에 들어 있어 추가 인자가 필요 없다.

`error` 계산에 `capacity` 를 빼면 안 되는 이유

조달로 용량이 늘면 같은 트래픽을 더 적은 배분으로 감당할 수 있으므로 이상 배분이 달라진다. 빠뜨리면 조달한 스텝의 정책이 부당하게 낮은 점수를 받고, ⑤가 "조달했더니 정책이 나빠졌다"고 잘못 학습한다.

⚠ 요청값과 적용값을 둘 다 남겨야 하는 이유

호출 순서상 `record_decision` 이 `apply_allocation` 보다 먼저다. 따라서 ④에는 에이전트가 요청한 배분만 기록되고, 액추에이터가 실제로 적용한 값(`normalized`)과 깎인 양(`delta`)은 어디에도 남지 않는다.

그런데 `error` 는 **적용값 기준**으로 계산된다. 기록(요청 0.70)과 채점(적용 0.49)의 기준이 달라 사후 분석에서 "정책이 나빴나, 액추에이터가 막았나" 를 구분할 수 없다.

계약의 예시가 정확히 그 경우다. 정책은 URLLC 0.70을 요청했고 이상값은 0.418이었다 — **요청 기준 `error`는 0.316, 적용 기준은 0.086.** 4배 차이다. ⑤가 둘 다 기록하면 이 구분이 가능해진다.

`observed_violations` 를 남기는 이유

MTTR 때문이다. ④가 가진 것은 `record_decision` 시점의 `obs_t` 뿐이라 에피소드 마지막 관측이 유실된다. ⑤가 채점하며 남기면 모든 스텝의 위반 시계열이 복원된다.

`ideal_allocation` 을 반환하는 이유

에이전트가 "얼마나 빡나갔나"를 숫자가 아닌 배분으로 볼 수 있다. 예시의 경우 실제로 필요했던 건 URLLC 물뿔(0.70)이 아니라 **균형에 가까운 배분**이었다. 배분보다 총량이 부족했던 상황이며, 에이전트가 ③의 조달을 고려해야 한다는 신호다.

축소 보정(`effective`)이 필요한 이유

없으면 첫 성공 1회로 신뢰도가 1.0이 되어 에이전트가 그 정책에 고착된다. `dqn` 의 `n=6` 처럼 표본이 적으면 `effective` 가 0.5 쪽으로 당겨진다.

신뢰도 종합을 에이전트만 하는 이유

②는 무상태라 ⑤를 알 수 없고, ②가 ⑤를 호출하면 서버 간 직접 호출이 된다. 내재적(②)과 경험적(⑤)은 다른 종류의 값이고, 둘을 합치는 $\sqrt{(\text{intrinsic} \times \text{effective})}$ 와 < 0.45 판단이 연구의 기여 지점이다 (`flow/data-chain.md`).

산출식

담당 B · claude/build/formulas.md

내재적 신뢰도 · 경험적 신뢰도 · 도구 설명 수위

6. 미결 사항 해소안

6.1 내재적 신뢰도 산출식 (§9 3번)

정책	산출식	근거
rule_based	$0.50 + 0.30 \cdot \min(1, m_{in})$ $m_{in} = \min_i \setminus u_i - \theta_i / \theta_i$	이용률이 임계값에 바짝 붙어 있으면 규칙의 이산 분기가 자의적 → 확신 낮음. 여유가 크면 분기가 명확. 범위 [0.50, 0.80]
lstm_forecast	$\exp(-3 \cdot \bar{e}) \cdot 1[in_distribution]$ $\bar{e} = \text{최근 5회 error의 EMA}$ (§4 ⑤ 정의)	모델에 추론 시 dropout이 없어 분산 추정이 불가. 최근 실적이 유일하게 값싼 대리 지표. $\bar{e}=0.1 \rightarrow 0.74$, $\bar{e}=0.3 \rightarrow 0.41$
dqn	$\sigma(Q_{(1)} - Q_{(2)}) \cdot 1[in_distribution]$	상위 2개 Q값 마진. 2차

rule_based의 하한을 0.50으로 잡은 것은 의도적이다. **항상 가용한 안전 기본값**이므로 다른 정책이 전부 실패해도 이것 하나는 에스컬레이션 문턱($\tau=0.45$) 위에 있다. 이 상수가 곧 "언제 사람을 부를 것인가"의 하한을 정의한다.

첫 5스텝은 $\bar{e}$ 표본이 없다. lstm_forecast는 status="unavailable" (history < 10)이므로 자연히 배제된다 — 별도 처리 불필요.

6.2 경험적 신뢰도 갱신식 (§9 4번)

```
r ← (1 - α)·r + α·s      α = 0.2,  s = 1 if sla_met else 0
n ← n + 1
```

```
effective = (r·n + r₀·m) / (n + m)      r₀ = 0.5,  m = 5
```

- $\alpha = 0.2$ — 반감기 약 3스텝. 시나리오가 전환되는 mixed (120스텝, 5회 전환)에서 전환 후 5~10스텝 내에 신뢰도가 따라붙는다. α 가 작으면 mixed에서 자기 개선이 안 보이고, 크면 잡음을 학습한다.
- **축소 항 (m=5)** — n이 작을 때 r이 0 또는 1로 튀는 것을 막는다. 첫 성공 1회로 신뢰도 1.0이 되면 에이전트가 그 정책에 고착된다.
- **에이전트의 종합** (분리 설계서 §2.2):

```
combined = sqrt( intrinsic × effective )      # 기하평균
escalate if combined <  $\tau$ ,     $\tau = 0.45$       # 사전 확정
```

기하평균을 쓰는 이유: 한쪽이 낮으면 전체가 낮아진다. 산술평균은 "이번엔 확신 없지만 평소 잘 맞으니 간다"를 허용하는데, 그건 에스컬레이션이 필요한 바로 그 상황이다.

$\tau = 0.45$ 고정. 실험 전에 확정하고 절대 조정하지 않는다. 조정하면 개입 횟수를 원하는 대로 만들 수 있어 결과가 무의미해진다. τ 민감도는 **사후 분석**으로 별도 보고한다 (`decisions.json` 에 `combined` 를 기록해 두었으므로 재실행 없이 τ 를 바꿔 곡선을 그릴 수 있다).

에피소드 간 신뢰도 유지 여부 — 두 모드를 모두 지원하고 실험 변수로 삼는다.

모드	<code>reliability.json</code>	보이는 것
<code>cold</code>	시나리오마다 초기화	단일 에피소드 내 적응
<code>warm</code>	전체 실행에 걸쳐 누적	장기 자기 개선 (논문의 §5 주장)

주 결과는 `warm`. `cold` 는 대조군.

6.3 도구 설명 수위 (§9 8번)

분리 설계서의 지적이 정확하다. MCP는 도구 설명을 LLM 컨텍스트에 그대로 넣으므로, "평시 효율 우수" 는 사실상 프롬프트다.

두 벌을 `mcp/policy/descriptions.py` 에 두고 환경변수로 전환한다.

```
DESC = {
  "minimal": {
    "rule_based": "임계값 기반 배분. 상황 라벨을 입력으로 받는다.",
    "lstm_forecast": "시계열 모델 기반 배분. 관측 이력 10스텝 필요.",
    "dqn": "강화학습 정책 기반 배분.",
  },
  "advisory": {
    "rule_based": "이율을 임계값 기반. 항상 가용, 보수적.",
    "lstm_forecast": "시계열 예측 기반. 평시 효율 우수.",
    "dqn": "강화학습 정책. 학습 분포 내에서 우수.",
  },
}
MODE = os.environ.get("SLICE_DESC_MODE", "minimal")
```

`minimal` 판은 무엇을 하는지와 전제조건만 말하고 언제 쓰라는 말을 하지 않는다. `lstm_forecast` 의 "이력 10스텝 필요" 는 조언이 아니라 호출 규약이므로 남긴다.

주 실험은 `minimal` 로 돌린다. `advisory` 는 비교 실행으로 붙인다.

- 두 조건의 정책 선택 분포가 비슷 → 에이전트가 관측으로 판단했다는 증거
- 크게 다름 → 설명에 이끌렸다는 증거이며, 이것도 **보고할 가치가 있는 결과**다 (MCP 기반 에이전트 설계의 일반 교훈)

어느 쪽이든 §9 8번이 위험에서 기여로 바뀐다. `desc_mode` 를 `decisions.json` 의 `config` 에 기록한다.

0단계 — 모델 적재 검증

담당 B · claude/build/step0.md

TF 2.15.1 고정 · 검증 6항목 · 실패 시 분기

7. 0단계 — 모델 적재 검증

나머지 전부의 전제다 (분리 설계서 §7-§9 0번). 정정 I 때문에 단순 설치보다 손이 더 간다.

```
py -3.10 -m venv .venv310
.venv310/Scripts/python -m pip install "tensorflow==2.15.1" numpy pandas fastmcp pydantic
.venv310/Scripts/python tools/step0_verify_models.py
```

tools/step0_verify_models.py 가 확인할 것 — 적재 성공만으로는 통과가 아니다.

#	확인	실패 시
1	load_model("models/lstm_model_20250601_011235") 성공	TF 버전 재확인 → TFSMLayer → 재학습
2	입력 shape가 (1, 10, 11) 을 받는가	create_feature_vector 차원 재확인
3	출력이 3차원이고 합 ≈ 1 , 각 항 $\in [0,1]$	스케일러 필요 → 재학습 (정정 I)
4	동일 입력 2회 → 동일 출력	비결정 레이어 존재 → 신뢰도 산출식 변경
5	load_model("models/dqn_model_20250601_012010") + softmax 합 ≈ 1	classify_demand 제외
6	dqn_training_data/*.csv 컬럼별 min/max → policy/ranges.json	in_distribution 판정 불가

3번이 진짜 관문이다. lstm_model_*/ 에 스케일러 파일이 없다 (models/lstm_single/ 에만 X_scaler.npy 존재). 적재는 되는데 출력이 [-3.2, 8.1, 0.4] 같으면 정규화 전제가 다른 것이고, 복원할 스케일러가 없으므로 재학습이다. 이 경우 lstm_forecast 의 작업량이 中~大로 되돌아간다.

분기 계획

3번 결과	1차 범위
통과	rule_based + lstm_forecast (분리 설계서 §5 권고대로)
실패	rule_based 단독으로 1차 진행

rule_based 단독이어도 논문은 성립한다. 정정 E 덕분에 "동일한 규칙 엔진에 상황 라벨만 사람 → 에이전트로 교체" 라는 가장 깨끗한 통제 실험이 되고, 에스컬레이션(신뢰도 기반 선택적 개입)은 정책 개수와 무관하게

돌아간다. 잃는 것은 "정책 선택" 측 하나뿐이다.

→ §9 1번(DQN 1차 포함 여부)의 답: 포함하지 않는다. 0단계 결과와 무관하게 결정 가능하다.

서버별 추출 대상 코드

담당 B · `claude/build/extract.md`

기존 코드의 어느 함수를 어느 서버로 가져오고 무엇을 고치는가. `mcp/` 는 **추출(copy-and-adapt)**이지 `import`가 아니다.

① `slice-observe`

추출 대상 (`ml_orchestrator_demo.py` → `mcp/observe/env.py`)

함수	원본 위치	조치
<code>generate_traffic()</code>	:286	<code>datetime.now()</code> → <code>sim_time</code> , <code>np.random</code> → <code>self.rng</code> (정정 F)
<code>update_utilization()</code>	:467	그대로
<code>create_feature_vector()</code>	:490	동일 치환
<code>thresholds</code> [0.9, 1.2, 0.8]	:174	그대로
위반 판정 <code>util > thresholds</code>	:563	그대로
<code>__init__</code> 상태	:162	<code>output_dir</code> 생성· <code>predictor</code> 적재 제거
<code>run()</code> 루프	:840	폐기 — <code>step()</code> 으로 대체
<code>visualize_*</code>	:643 , :786	폐기
5번째 스텝 벤더 특수처리	:580~	폐기

도구

```
get_observation() -> Observation
step(n: int = 1) -> {"steps_advanced": int, "observation": Observation}
apply_allocation(embb: float, urlc: float, mmtc: float) -> ApplyResult # §3.2
get_history(n: int = 10) -> {"features": [[float × 11], ...], "columns": [str × 11]}
add_capacity(slice_type: str, amount: float,
              expires_at_step: int, slice_id: str) -> CapacityState # 정정 K
reset(run_id: str, scenario: str = "normal", seed: int = 0) -> {"observation": Observation}
```

Observation 스키마 — `is_emergency` 계열 3개는 부재한다.

```
{
  "step": 12,
  "sim_time": {"hour_of_day": 11, "day_of_week": 0, "is_weekend": false},
  "traffic": {"embb": 0.52, "urllc": 0.61, "mmtc": 0.19},
  "allocation": {"embb": 0.40, "urllc": 0.40, "mmtc": 0.20}, # 현재 적용 중
  "utilization": {"embb": 1.30, "urllc": 1.52, "mmtc": 0.95}, # traffic / allocation
  "thresholds": {"embb": 0.9, "urllc": 1.2, "mmtc": 0.8}, # 노출한다 (정답 아님)
  "violations": {"embb": true, "urllc": true, "mmtc": true},
  "client_count": 0.63,
  "bs_count": 0.48
}
```

`thresholds` 를 노출하는 이유: 사람이 설정하던 값(개입 지점 3번)이고, 이것 모르면 에이전트가 `violations` 의 의미를 해석할 수 없다. 상황 정답과는 무관하다.

get_history() 컬럼 순서 고정 — `create_feature_vector()` (:513~526)와 정확히 동일해야 LSTM이 맞는다. 문자열로 함께 반환해 ②가 재유도하지 않게 한다.

```
[traffic_load, hour_of_day, day_of_week,
 embb_alloc, urllc_alloc, mmtc_alloc,
 embb_util, urllc_util, mmtc_util,
 client_count, bs_count]
```

정답 기록 — `step()` 내부에서 `runs/{run_id}/truth.jsonl` 에 append (\$5.0).

```
{"step": 12, "is_emergency": true, "is_special_event": false, "is_iot_surge": false}
```

`_get_ground_truth()` 를 파이썬 함수로 두는 것보다 파일 append가 낫다. ①은 별도 프로세스이므로 평가 스크립트가 함수를 부를 수 없고, 파일이면 `@mcp.tool` 데코레이터가 없는 것만으로 누출 차단이 증명된다.

mixed 시나리오 변환 — `test_scenarios.py:92` 의 `dynamic_changes` 는 초 단위(`interval=0.5`)다. 스텝 단위로 옮긴다.

원본 time (초)	스텝	이벤트
10 → 25	20 → 50	<code>special_event</code> on/off
30 → 45	60 → 90	<code>emergency</code> on/off
50 →	100 →	<code>iot_surge</code> on

`duration=60, interval=0.5` → **120스텝**. 시나리오 정의는 `test_scenarios.py:54-102` 를 그대로 옮기되 변환표를 `env.py` 에 상수로 둔다.

작업량: 中 (분리 설계서와 동일. 정정 F의 5줄이 추가되지만 `run()` 시각화 폐기로 상쇄)

② slice-policy

도구

```
list_policies() -> [PolicyInfo]

propose_allocation(
    policy: str,
    observation: dict,
    situation: str,          # ← 필수. "normal"|"emergency"|"special_event"|"iot_surge"
    history: list = None
) -> {
    "policy": str,
    "allocation": {"embb", "urllc", "mmtc"} | None,
    "confidence": float,
    "in_distribution": bool,
    "status": "ok" | "unavailable" | "error",      # ← 정정 H
    "reason": str,
    "rationale": str
}

compare_policies(observation, situation, history=None) -> [위와 동일 × N]

classify_demand(observation) -> {
    "dominant": "eMBB"|"URLLC"|"mMTC",
    "probabilities": {...}
}
```

situation 은 **rule_based**에만 쓰인다. **lstm_forecast**·**dqn**은 관측 벡터만 본다. 그래도 시그니처를 통일하는 이유: 에이전트가 **상황 판단을 매 결정마다 명시하게 강제**하고, 그 값이 ④에 기록되어 **perception_accuracy** 계산의 입력이 되기 때문이다. **situation**이 없으면 에이전트가 무엇이라 판단했는지 측정할 방법이 없다.

정책별 구현

정책	출처	조치	가중치
rule_based	:422	<code>self.is_emergency</code> → 인자 <code>situation</code> (정정 E). 평활·클립 제거(→①)	불필요
lstm_forecast	<code>lstm_predictor.py:426</code>	폴백 제거(정정 H). 평활·클립 제거	있음 (2.9 MB)
dqn	<code>dqn_agent.py</code>	2차. 데이터 재생성 선행	없음
classify_demand	<code>dqn_classifier.py:359</code> <code>classify()</code>	그대로	있음 (471 KB)

모델 적재는 서버 기동 시 1회. `propose_allocation` 호출마다 적재하면 스텝당 수 초가 든다. 적재 실패 시 서버는 죽지 않고 해당 정책만 `status="unavailable"`로 응답한다 — 실패를 감추지 않으면서 ③④⑤ 실험은 계속 돌게 한다.

in_distribution 판정 — **lstm_forecast**는 `get_history()`의 11차원이 학습 범위 안인지 본다. 학습 데이터 통계가 없으므로 0단계에서 `dqn_training_data/*.csv`의 컬럼별 min/max를 뽑아 `policy/ranges.json`으로

고정한다. 범위 밖 차원이 하나라도 있으면 `false`.

작업량: `rule_based` 小 / `lstm_forecast` 小~中 / `classify_demand` 小 / `dqn` 大(2차)

③ `slice-market`

정정 G에 따라 모델 A 단일화. 추출 대상 (`engine.py` → `mcp/market/scoring.py`)

함수	위치	조치
<code>_calculate_criteria_weights()</code>	:252	그대로
<code>_score_latency/bandwidth/reliability()</code>	:328~417	그대로
<code>_calculate_qos_match()</code>	:592	그대로
<code>_calculate_price_score()</code>	:628	그대로
<code>score_vendor_offering()</code>	:719	<code>async</code> 제거, <code>logger</code> 정리
<code>get_score_breakdown()</code>	:819	그대로 (3중 중복 중 이것만)
<code>query_vendors()</code> / <code>find_matching_offerings()</code>	:95 / <code>registry.py</code> :349	사용 안 함 (정정 G)
<code>_advanced_score_offer()</code>	:185	사용 안 함
<code>_update_vendor_rating()</code>	<code>loop.py</code> :218	24시간 창 → 스텝 창으로 변경

도구

```
list_offerings(slice_type=None, region=None) -> [Offering]
score_offerings(slice_type, qos_requirements) -> [ScoredOffering]      # 점수 내림차순
explain_score(vendor_id, slice_type, qos_requirements) -> ScoreBreakdown
procure(vendor_id, slice_type, qos_requirements, duration_steps) -> {"slice_id", "status"}
update_rating(vendor_id, outcome) -> {"vendor_id", "rating", "delta"}
```

`duration_hours` → `duration_steps`. ①이 가상 시계를 쓰므로 시간 단위를 섞지 않는다.

`update_rating` 갱신식 — `loop.py`의 24시간 창을 스텝 창으로 옮기되, 단순화한다.

```
rating ← clip(rating + δ, 1.0, 5.0)
δ = +0.05 (sla_met = true)
   = -0.20 (sla_met = false)
```

위반에 4배 가중. 5곳의 초기 레이팅이 4.5~4.9로 좁게 몰려 있어 작은 δ 로는 순위가 안 바뀐다.
`engine.py`:792에서 `rating/5.0`이 총점의 10~20%를 차지하므로, 0.2 하락 = 총점 0.4~0.8점 하락. 60~120스텝 실험에서 순위 역전이 관측 가능한 크기다.

부트스트랩 (`tools/bootstrap_vendors.py`, 1회 실행)

5G-Marketplace/data/vendors.json → data/vendors.json
+ 각 벤더에 "regions": [...] 추가 (location 문자열에서 유도, 5줄)

작업량: 小 — 정정 G로 `await` 수정도 불필요해져 더 줄었다. 1번으로 만든다.

④ slice-audit

정정 D에 따라 정답 의존 지표를 제외한다.

```
record_decision(
    step: int, observation: dict,
    situation: str,                # 에이전트의 상황 판단 ← perception 측정의 입력
    chosen_policy: str, allocation: dict,
    confidence: dict,              # {"intrinsic": .., "empirical": .., "combined": ..}
    rationale: str
) -> {"decision_id": str}

record_escalation(step, observation, situation, reason, confidence) -> {"escalation_id": str}

get_decisions(n: int = 50) -> [Decision]

get_metrics(window: int = None) -> {
    "steps": int,
    "interventions": int,
    "autonomous_rate": float,
    "sla_violations": int,
    "mean_utilization": {...},
    "policy_usage": {...},
    "mttr": float
    # perception_accuracy · escalation_precision ← 제외. eval/score.py 담당
}
```

`situation` 을 기록하는 것이 ④의 가장 중요한 역할이다. 이것과 `truth.jsonl` 의 조인이 논문의 핵심 지표를 만든다.

`decision_id` = `f"{run_id}-{step:04d}"`. UUID보다 낫다 — 정렬 가능하고, ⑤가 `step` 으로 `truth.jsonl` 과 대조할 수 있고, 사람이 로그를 읽을 수 있다.

mttr 산출 (정답 불필요): `violations` 가 하나라도 `true` 인 첫 스텝 → 전부 `false` 가 된 스텝까지의 길이. 복구되지 않은 채 끝난 구간은 제외하고 별도로 `unresolved` 카운트로 보고한다. 조용히 빼면 MTTR이 좋아 보인다.

작업량: 小~中

⑤ slice-feedback

```
report_outcome(decision_id: str, observed: dict) -> {
    "sla_met": bool,
    "policy": str,
    "error": float,
    "vendor_id": str | None,          # 있으면 에이전트가 ③.update_rating() 중계
    "reliability_after": float
}

get_reliability_table() -> {
    "rule_based": {"reliability": 0.91, "n": 120, "effective": 0.88},
    "lstm_forecast": {"reliability": 0.84, "n": 95, "effective": 0.82},
    "dqn": {"reliability": 0.42, "n": 60, "effective": 0.45}
}
```

error 정의 — 분리 설계서의 "기대 대비 실제 오차"를 확정한다.

```
이상 배분  a*_t = normalize( traffic_t / thresholds )    # 위반이 딱 없어지는 배분
error      = L1( a_chosen , a*_t ) / 2                  # [0, 1]
```

a* 는 관측에서 계산되고 정답을 쓰지 않는다. 정책 간 비교가 가능하고 ②의 LSTM 신뢰도 산출에도 재사용된다 (§6.1).

sla_met = **observed.violations** 의 세 값이 모두 **false** .

작업량: 中

구현 순서와 미결 사항

담당 B · claude/build/order.md

단계별 완료 판정 · ③ 서버 골격 · 남은 결정

8. 구현 순서와 체크포인트

분리 설계서 §7을 유지하되, 각 단계의 완료 판정 기준을 붙인다.

순서	대상	완료 판정	작업량
0	모델 적재 검증	§7의 6개 항목 통과. <code>ranges.json</code> 생성	小
1	③ <code>slice-market</code>	에이전트가 <code>score_offerings</code> 호출 → 5개 벤더 점수 수신. MCP 배선 검증	小
2	① <code>slice-observe</code>	동일 시드 2회 실행 → <code>truth.jsonl</code> 바이트 단위 동일 (정정 F 검증)	中
3	④ <code>slice-audit</code>	<code>get_metrics()</code> 반환 필드에 정답 의존 지표 부재 확인 (정정 D)	小~中
4	② <code>slice-policy</code>	<code>situation</code> 인자로 배분이 바뀜(정정 E). LSTM 실패 시 <code>status="unavailable"</code> 반환(정정 H)	中
5	⑤ <code>slice-feedback</code>	<code>warm</code> 모드 120스텝에서 <code>reliability.json</code> 의 값이 실제로 움직임	中
6	<code>eval/score.py</code>	<code>truth.jsonl</code> × <code>decisions.json</code> 조인 → <code>perception_accuracy</code> 산출	小
7	베이스라인 실행	비교군 4개 × 시나리오 5개 × 시드 3개 = 60회	中
8	② <code>dqn</code> 추가	선택. 데이터 재생성 선행	大

2단계의 완료 판정이 가장 중요하다. 동일 시드 재현이 안 되면 이후 모든 비교가 무의미하다. 여기서 반드시 멈추고 확인한다.

1단계(③)를 먼저 하는 이유는 분리 설계서와 같다 — **MCP 배선 자체를 검증**하는 것이 목적이지 ③이 중요해서가 아니다. FastMCP 서버 하나, stdio 연결 하나, 도구 호출 하나가 왕복하면 나머지는 같은 패턴의 반복이다.

③ 서버 골격 (1단계 산출물 예시)

```
# mcp/market/server.py
from fastmcp import FastMCP
from mcp.common.store import read_json, write_json_atomic
from mcp.common.paths import VENDORS
from mcp.market.scoring import score_vendor_offering, get_score_breakdown

mcp = FastMCP("slice-market")

@mcp.tool()
def score_offerings(slice_type: str, qos_requirements: dict) -> list[dict]:
    """주어진 slice_type과 QoS 요구에 대해 등록된 모든 벤더를 점수화한다."""
    vendors = read_json(VENDORS)
    out = [{"vendor_id": v["id"], "name": v["name"],
            "score": score_vendor_offering(v, slice_type, qos_requirements),
            "rating": v["rating"]}
           for v in vendors if slice_type in v.get("offerings", {})]
    return sorted(out, key=lambda x: x["score"], reverse=True)

@mcp.tool()
def update_rating(vendor_id: str, outcome: dict) -> dict:
    """결과 보고를 반영해 벤더 레이팅을 갱신한다. ⑤의 보고를 에이전트가 중계한다."""
    vendors = read_json(VENDORS)
    delta = 0.05 if outcome.get("sla_met") else -0.20
    for v in vendors:
        if v["id"] == vendor_id:
            v["rating"] = max(1.0, min(5.0, v["rating"] + delta))
            write_json_atomic(VENDORS, vendors)
            return {"vendor_id": vendor_id, "rating": v["rating"], "delta": delta}
    return {"error": f"unknown vendor: {vendor_id}"}

if __name__ == "__main__":
    mcp.run()
```

도구 `docstring`이 곧 LLM 프롬프트라는 점을 여기서부터 의식한다 (§6.3).

9. 남은 미결 사항

분리 설계서 §9 중 이 문서에서 해소되지 않은 것.

#	항목	상태
0	TF 적재 검증	\$7로 절차 확정. 실행 대기
1	DQN 1차 포함 여부	해소 — 포함하지 않음 (\$7)
2	DQN 상태의 <code>is_emergency</code> 차원	2차로 이월. 에이전트 추론값 주입이 기본안
3	내재적 신뢰도 산출식	해소 (\$6.1)
4	경험적 신뢰도 갱신식	해소 (\$6.2)
5	오케스트레이터 LLM 선택·비용	미해소. 스텝당 3~5회 도구 호출 × 120스텝 × 60회 실행 ≈ 2~3만 호출. 모델 선정 전 토큰 예산 산정 필요
6	문헌 조사	미해소 (3단계)
7	Dashboard API 재활용	미해소 (6단계)
8	도구 설명 수위	해소 — 실험 변수로 승격 (\$6.3)
9	에이전트 프롬프트 배경 지식 범위	미해소. \$6.3과 같은 문제가 프롬프트에도 있음. <code>decisions.json</code> 에 프롬프트 해시를 기록해 사후 추적 가능하게 할 것
10	조달의 환경 반영 방식	해소 — ①에 용량 배수 도입 (정정 K)
11	조달 비용 억제 장치	해소 — 예산 대신 상한·만료 + 비용을 논문 지표로 보고

5번이 다음 병목이다. 2~3만 회의 도구 호출은 비용과 시간 모두에서 무시할 수 없다. 2단계 개발계획서에서 다음을 정해야 한다.

- 모델 선정 (로컬 Ollama vs API)
- 시드 3개가 맞는지, 시나리오 5개를 다 돌릴지
- 스텝당 호출 횟수 상한 (도구 호출 루프가 발산하지 않도록)

9번은 8번과 같은 위험이다. 도구 설명을 최소판으로 만들어도 시스템 프롬프트에 "URLLC 이용률이 높으면 비상 상황일 수 있다" 라고 적으면 같은 문제가 된다. 프롬프트도 최소판/조언판 두 벌을 준비하고 `config` 에 기록한다.

B · 모델 · 정책 — 작업 목록

담당 B · claude/team/roles.md

3. B — 모델 · 정책

한 줄 요약: TensorFlow를 되살리고, 정책을 나란히 세우고, 결과에서 배우게 한다.

3.1 작업 순서 — 0단계부터, 그 다음 ③

0단계가 전체 프로젝트의 전제입니다 (설계서 §7). 결과에 따라 B 본인의 작업량이 중에서 大로 바뀌고, C의 프롬프트 설계도 달라집니다. Day 1 오전에 끝내고 즉시 공유하세요.

B-0. 모델 적재 검증 (Day 1 오전) ⚠

```
py -3.10 -m venv .venv310
.venv310/Scripts/python -m pip install "tensorflow==2.15.1" numpy pandas fastmcp pydantic
.venv310/Scripts/python tools/step0_verify_models.py
```

버전 고정이 핵심입니다. TF 2.16+는 Keras 3가 기본인데, Keras 3의 `load_model()` 은 SavedModel 디렉터리를 읽지 못합니다. 가중치가 `saved_model.pb + variables/` 형식이라 `pip install tensorflow` 만 하면 최신이 깔려 그대로 실패합니다.

`tools/step0_verify_models.py` 가 확인할 것 — 적재 성공만으로는 통과가 아닙니다:

#	확인	실패 시
1	<code>load_model("5G-Network-Slicing/models/lstm_model_20250601_011235")</code>	TF 버전 재확인 → <code>TFSMLayer</code> → 재학습
2	입력이 (1, 10, 11) 을 받는가	피쳐 차원 재확인
3	출력 3차원, 합 ≈ 1 , 각 항 $\in [0,1]$	스케일러 필요 → 재학습
4	동일 입력 2회 → 동일 출력	비결정 레이어 → 신뢰도 식 변경
5	<code>dqn_model_20250601_012010</code> 적재 + softmax 합 ≈ 1	<code>classify_demand</code> 제외
6	<code>dqn_training_data/*.csv</code> 컬럼별 min/max → <code>mcp/policy/ranges.json</code>	<code>in_distribution</code> 판정 불가

3번이 진짜 관문입니다. `lstm_model_*/` 에는 스케일러 파일이 없습니다 (`models/lstm_single/` 에만 `X_scaler.npy` 존재). 적재는 되는데 출력이 `[-3.2, 8.1, 0.4]` 같으면 정규화 전제가 다른 것이고, 복원할 스케일러가 없으므로 재학습입니다.

결과를 3명 모두에게 즉시 공유하세요:

3번 결과	1차 범위	C에 미치는 영향
통과	<code>rule_based</code> + <code>lstm_forecast</code>	"정책 선택" 프롬프트 필요
실패	<code>rule_based</code> 단독	정책 선택 축 제거, 상황 인지 + 에스컬레이션에 집중

`rule_based` 단독이어도 논문은 성립합니다. 정정 E 덕분에 "동일한 규칙 엔진에 상황 라벨만 사람 → 에이전트로 교체" 라는 가장 깨끗한 통제 실험이 되고, 에스컬레이션은 정책 개수와 무관합니다.

B-1. ③ `slice-market` (Day 1 오후 ~)

가장 먼저 완성되는 서버입니다. 목적은 ③ 자체가 아니라 MCP 배선이 실제로 도는지 확인하는 것입니다. C의 목 서버를 진짜로 교체하는 첫 대상입니다 (\$7 M1).

`engine.py` 에서 가져올 것 → `mcp/market/scoring.py` :

함수	위치	조치
<code>_calculate_criteria_weights()</code>	:252	그대로
<code>_score_latency/bandwidth/reliability()</code>	:328~417	그대로
<code>_calculate_qos_match()</code>	:592	그대로
<code>_calculate_price_score()</code>	:628	그대로
<code>score_vendor_offering()</code>	:719	<code>async</code> 제거
<code>get_score_breakdown()</code>	:819	3중 중복 중 이것만
<code>query_vendors()</code> / <code>find_matching_offerings()</code>	:95 / <code>registry.py</code> :349	사용 안 함

벤더 데이터가 두 벌이고 호환되지 않습니다 (정정 G).

	모델 A (채택)	모델 B (폐기)
파일	<code>5G-Marketplace/data/vendors.json</code>	<code>5G-Marketplace/data/vendor_registry/*.json</code>
구조	리스트 5개, 중첩 offerings	dict, offering_id 평탄화 (6건)
<code>rating</code>	있음 (4.5~4.9)	없음

`engine.py`:792 의 점수 계산은 모델 A만 읽습니다. 모델 B를 쓰면 rating이 항상 기본값 3.0으로 떨어져 ⑤의 레이팅 갱신이 점수에 전혀 반영되지 않습니다.

→ `tools/bootstrap_vendors.py` 로 모델 A를 `data/vendors.json` 에 복사하면서 `regions` 필드만 추가 (벤더 5곳 × 1줄). 부수 효과로 `engine.py`:115 의 `await` 누락 버그가 고칠 필요 없이 사라집니다 (해당 경로 미사용).

도구 5개:

```
list_offerings(slice_type=None, region=None) -> [Offering]
score_offerings(slice_type, qos_requirements) -> [ScoredOffering] # 점수 내림차순
explain_score(vendor_id, slice_type, qos_requirements) -> ScoreBreakdown
procure(vendor_id, slice_type, qos_requirements, duration_steps, current_step)
    -> {"slice_id", "status", "capacity_gain", "expires_at_step", "cost_total"}
update_rating(vendor_id, outcome) -> {"vendor_id", "rating", "delta"}
```

`duration_hours` → `duration_steps`. ①이 가상 시계를 쓰므로 단위를 섞지 않습니다.

W2 — `current_step` 을 받아야 합니다. ③은 ①과 별개 프로세스라 시뮬레이션 시각을 모르는데 `expires_at_step = current_step + duration_steps` 를 반환해야 합니다.

W3 — 비용 단위를 환산합니다. `cost` 는 시간당, `duration_steps` 는 스텝입니다.

```
cost_total = cost × duration_steps × (MINUTES_PER_STEP / 60) = 250 × 10 × 0.25 = 625.0
```

그냥 곱하면(2500.0) **비용이 4배 부풀어** `procurement_cost_total` 지표가 무의미해집니다.

W4 — `vendor_id` 를 반환값에 포함하세요. ④에 기록되어야 ⑤가 결과 보고 시 되찾습니다.

B-2. ② `slice-policy`

```
list_policies() -> [PolicyInfo]
propose_allocation(policy, observation, situation, history=None) -> PolicyProposal
compare_policies(observation, situation, history=None,
    recent_errors=None) -> [PolicyProposal] # W6: 정책별 딕셔너리
classify_demand(observation) -> {"dominant", "probabilities"}
```

두 가지가 이 서버의 전부입니다.

(1) `situation` 을 필수 인자로 받습니다 (정정 E). `update_allocation_rule_based()` (:422)는 `self.is_emergency` 를 직접 읽습니다 (:429). 이걸 그대로 옮기면 ②가 정답을 들고 있게 되고, ②는 무상태라 가질 수도 없습니다.

```
# 원본 :429
if self.is_emergency:           → if situation == "emergency":
elif self.is_special_event:     → elif situation == "special_event":
elif self.is_iot_surge:         → elif situation == "iot_surge":
```

기본값을 주지 마세요. `"normal"` 을 기본값으로 두면 에이전트가 생략했을 때 조용히 평시로 처리되어 미탐이 측정에서 사라집니다.

(2) 폴백 금지 (정정 H). `update_allocation_ml()` (:341)은 실패 시 `update_allocation_rule_based()` 를 조용히 호출합니다 (:414, :419). 그대로 두면 `propose_allocation(policy="lstm_forecast")` 가 **rule_based** 결과를 반환하면서 **"policy": "lstm_forecast"** 라고 답합니다. ④의 `policy_usage` 가 통째로 거짓이 되고 ⑤가 잘못된 대상을 학습합니다.

```
{"policy": "lstm_forecast", "allocation": None, "confidence": 0.0,
 "status": "unavailable", "reason": "history_insufficient: 4 < 10"}
```

폴백 여부는 에이전트가 결정합니다. 그게 곧 "정책 선택"이고 이 연구의 주제입니다.

V4 — `recent_error` 를 인자로 받습니다. `lstm_forecast` 의 신뢰도 $\exp(-3 \cdot \bar{e})$ 에서 $\bar{e}$ 는 누적 오차인데, ②는 무상태이고 ⑤를 호출할 수 없습니다. ②가 원리적으로 알 수 없는 값을 식에 넣어 두었던 것입니다.

```
⑤.get_reliability_table() → {"lstm_forecast": {..., "recent_error": 0.112}}
                               | 에이전트가 중계
                               ↓
②.propose_allocation(policy="lstm_forecast", ..., recent_error=0.112)
```

②는 여전히 무상태입니다 — 누적값을 들고 있는 건 ⑤이고, ②는 받은 값을 식에 넣을 뿐입니다.
`recent_error` 가 `null` 이면 보수적 기본값 `0.5` 를 쓰고 `rationale` 에 그 사실을 적으세요.

일반 규칙: 무상태 서버의 산출식에 "최근 N회"."누적"."평균"이 들어가면 그 값은 반드시 인자여야 합니다.

신뢰도 산출식 (설계서 §6.1):

정책	식
<code>rule_based</code>	$0.50 + 0.30 \cdot \min(1, \min_i \ u_i - \theta_i\ / \theta_i) \rightarrow [0.50, 0.80]$
<code>lstm_forecast</code>	$\exp(-3 \cdot \bar{e}) \cdot 1[\text{in_distribution}]$, $\bar{e}$ = 최근 5회 <code>error</code> 의 EMA

`rule_based` 의 하한 0.50은 의도적입니다. 항상 가용한 안전 기본값이므로 다른 정책이 다 실패해도 이것 하나는 $\tau(0.45)$ 위에 있습니다.

모델 적재는 서버 기동 시 1회. 호출마다 적재하면 스텝당 수 초가 듭니다. 적재 실패 시 서버는 죽지 않고 해당 정책만 `status="unavailable"` 로 응답합니다.

도구 설명 2벌 — `mcp/policy/descriptions.py` (설계서 §6.3):

```
DESC = {
  "minimal": {"lstm_forecast": "시계열 모델 기반 배분. 관측 이력 10스텝 필요."},
  "advisory": {"lstm_forecast": "시계열 예측 기반. 평시 효율 우수."},
}
MODE = os.environ.get("SLICE_DESC_MODE", "minimal")
```

MCP는 도구 설명을 LLM 컨텍스트에 그대로 넣습니다. "평시 효율 우수" 는 사실상 "언제 이걸 골라라" 는 조언입니다. 주 실험은 `minimal`, `advisory` 는 비교 실행.

B-3. ⑤ `slice-feedback`

```
report_outcome(decision_id, observed) -> {"sla_met", "policy", "error", "vendor_id", "reliability_after"}
get_reliability_table() -> {"rule_based": {"reliability": ..., "n": ..., "effective": ...}}
```

`error` 정의 (설계서 §4):

```
a*_t = normalize(traffic_t / thresholds)      # 위반이 딱 없어지는 배분
error = L1(a_chosen, a*_t) / 2                # [0, 1]
```

정답을 쓰지 않고 관측만으로 계산됩니다. B-2의 LSTM 신뢰도에도 재사용됩니다.

갱신식 (§6.2):

```
r ← (1-α)·r + α·s          α = 0.2, s = sla_met
effective = (r·n + 0.5·5) / (n + 5)
```

축소 항이 없으면 첫 성공 1회로 신뢰도가 1.0이 되어 에이전트가 고착됩니다.

V5 — 요청값과 적용값을 둘 다 기록합니다. 호출 순서상 `record_decision` (3)이 `apply_allocation` (4)보다 먼저라, ④에는 **요청한 배분만** 남고 액추에이터가 실제 적용한 값은 어디에도 안 남습니다. 그런데 `error` 는 적용값 기준입니다.

```
요청 {0.20, 0.70, 0.10} → 평활 후 적용 {0.34, 0.49, 0.17}
이상 배분 {0.426, 0.418, 0.157}
```

```
error (적용 기준) = 0.086      ← ㉓가 계산하는 값
error (요청 기준) = 0.316     ← 4배 차이
```

기준이 다르면 "정책이 나뉘냐, 액추에이터가 막았냐" 를 구분할 수 없습니다. `report_outcome` 이 `applied_allocation · requested_allocation · actuator_delta` 를 함께 남기세요. `requested_allocation` 은 `decisions.json` 에서 `decision_id` 로 조회합니다.

`observed_violations` 도 남깁니다 — ④가 가진 건 `obs_t` 뿐이라 **에피소드 마지막 관측이 유실**되고, MTTR의 마지막 구간이 잘립니다.

V4 — `get_reliability_table()` 이 `recent_error` 를 함께 반환합니다. ②의 신뢰도 공급선입니다 (위 B-2 참조).

⑤는 `vendors.json` 에 직접 쓰지 않습니다. `vendor_id` 를 결과에 실어 보내면 에이전트가 ③의 `update_rating()` 을 호출합니다. `loop.py:330` 의 `self.ai_agent.update_vendor_ratings()` 같은 직접 호출은 MCP에서 불가능합니다.

에피소드 간 유지 모드 2개를 지원하세요 — 실험 변수입니다.

모드	<code>reliability.json</code>	보이는 것
<code>cold</code>	시나리오마다 초기화	단일 에피소드 내 적응
<code>warm</code>	전체 실행에 걸쳐 누적	장기 자기 개선 (주 결과)

3.2 완료 판정

#	판정
B-0	6개 항목 통과. <code>ranges.json</code> 생성. 결과를 3명에게 공유
B-1	에이전트가 <code>score_offerings</code> 호출 → 벤더 5개 점수 수신
B-2	같은 관측에 <code>situation</code> 만 바꾸면 배분이 달라진다. LSTM 실패 시 <code>status="unavailable"</code>
B-3	<code>warm</code> 120스텝에서 <code>reliability.json</code> 값이 실제로 움직인다
B-2	<code>recent_error=null</code> 과 <code>=0.3</code> 이 다른 <code>confidence</code> 를 낸다 (V4)
B-3	<code>report_outcome</code> 이 <code>applied</code> · <code>requested</code> · <code>actuator_delta</code> 를 모두 반환 (V5)

3.3 B의 함정

- `from tensorflow.keras...` 가 모듈 최상단에 있습니다 (`lstm_predictor.py:20` , `dqn_classifier.py:15`). TF가 없으면 import만으로 죽습니다. ② 서버는 TF 없이도 기동되어 `rule_based` 는 서빙해야 합니다 — 적재를 `try` 안에 넣고 실패를 상태로 들고 있으세요.
 - `requirements.txt` 에 `tensorflow`가 없습니다. 추가하되 `==2.15.1` 로 고정.
 - **평활·클립을 ②에 두지 마세요.** ①의 `apply_allocation()` 소관입니다. 양쪽에 다 있으면 0.7이 두 번 걸려 배분이 거의 안 움직입니다.
-